Simulasi Dampak Kebijakan Fiskal Daerah terhadap Kinerja Pembangunan Kabupaten/Kota di Jawa Timur

Notebook ini membangun **model simulasi skenario kebijakan** (bukan pure forecasting dan bukan klaim kausal murni) untuk 38 kabupaten/kota di Jawa Timur periode 2020–2024.

Tujuan

1. Menggabungkan data multi-folder menjadi panel tunggal kabupaten/kota–tahun.
2. Membangun model panel supervised ter-regularisasi dengan validasi berbasis grup wilayah.
3. Menjalankan simulasi tiga skenario kebijakan fiskal/ekonomi.
4. Menghasilkan output siap-join ke GeoJSON Jawa Timur untuk dashboard spasial.

Catatan penting metodologis

- Model ini untuk **policy scenario simulation** berbasis pola historis data.
- Hasil simulasi adalah **proyeksi kondisional** terhadap perubahan input kebijakan, bukan estimasi dampak kausal definitif.

```
import re
import warnings
from pathlib import Path
from collections import defaultdict

import numpy as np
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.model_selection import GroupKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 200)
pd.set_option("display.float_format", lambda x: f"{x:,.4f}")

ROOT = Path(".").resolve()
YEARS = list(range(2020, 2025))
```

```
def extract_year(text: str):
    match = re.search(r"(20\d{2})", str(text))
    return int(match.group(1)) if match else None


def clean_numeric(value):
    """Robust numeric parser for Indonesian locale and scientific not
    if pd.isna(value):
        return np.nan
    if isinstance(value, (int, float, np.number)):
        return float(value)

    raw = str(value).strip().replace(" ", "")
    if raw == "" or raw.lower() in {"nan", "none", "-", "n/a"}:
        return np.nan

    # Handle scientific notation with comma decimal, e.g., 2,30013E+1
    if re.search(r"[eE]", raw):
        raw = raw.replace(",", ".")
        try:
            return float(raw)
        except ValueError:
            return np.nan

    # Handle mixed separators
    if "," in raw and "." in raw:
        # Assume Indonesian style: dot thousand separator, comma deci
        if raw.rfind(",") > raw.rfind("."):
            raw = raw.replace(".", "").replace(",", ".")
        else:
            # Assume comma thousand separator
            raw = raw.replace(",", "")
    elif "," in raw:
        raw = raw.replace(",", ".")

    try:
        return float(raw)
    except ValueError:
        return np.nan


def normalize_text(text: str) -> str:
    text = "" if pd.isna(text) else str(text)
    text = text.strip().lower()
    text = re.sub(r"\s+", " ", text)
    return text


def format_region_name_from_pad_folder(folder_name: str) -> str:
    folder_name = folder_name.strip().lower()
    if folder_name.startswith("kab_"):
        base = folder_name.replace("kab_", "", 1)
        return "Kabupaten " + base.replace("_", " ").title()
```

```
    if folder_name.startswith("kota_"):
        base = folder_name.replace("kota_", "", 1)
        return "Kota " + base.replace("_", " ").title()
    return folder_name.replace("_", " ").title()


def build_region_reference(pad_root: Path):
    region_dirs = sorted([d for d in pad_root.iterdir() if d.is_dir()
    rows = []
    for idx, region_dir in enumerate(region_dirs, start=1):
        nama = format_region_name_from_pad_folder(region_dir.name)
        rows.append(
            {
                "kode_kabkota": f"JTM{idx:03d}",  # surrogate ID (buk
                "nama_kabkota": nama,
                "pad_folder": region_dir.name,
                "base_name": nama.replace("Kabupaten ", "").replace("
            }
        )

    region_ref = pd.DataFrame(rows)

    canonical_lookup = {
        normalize_text(row["nama_kabkota"]): row["nama_kabkota"]
        for _, row in region_ref.iterrows()
    }

    base_lookup = defaultdict(list)
    for _, row in region_ref.iterrows():
        base_lookup[normalize_text(row["base_name"])].append(row["nam

    return region_ref, canonical_lookup, base_lookup


def harmonize_region_name(raw_name, canonical_lookup, base_lookup):
    """Map names from heterogeneous files to canonical names based on
    name = normalize_text(raw_name)
    if not name:
        return np.nan

    if name in canonical_lookup:
        return canonical_lookup[name]

    starts_kab = name.startswith("kabupaten ")
    starts_kota = name.startswith("kota ")
    base = re.sub(r"^(kabupaten|kota)\s+", "", name)

    if base in base_lookup:
        candidates = base_lookup[base]
        if len(candidates) == 1:
            return candidates[0]
        if starts_kota:
            for c in candidates:
                if c.lower().startswith("kota "):
                    return c
```

```
        if starts_kab:
            for c in candidates:
                if c.lower().startswith("kabupaten "):
                    return c
        # default to Kabupaten when ambiguous raw name has no prefix
        for c in candidates:
            if c.lower().startswith("kabupaten "):
                return c
        return candidates[0]

    return np.nan


pad_root = ROOT / "pad"
assert pad_root.exists(), f"Folder PAD tidak ditemukan: {pad_root}"

region_ref, canonical_lookup, base_lookup = build_region_reference(pa
print(f"Jumlah wilayah dari folder pad/: {len(region_ref)}")
region_ref.head()
```

```
Jumlah wilayah dari folder pad/: 38
```

	kode_kabkota	nama_kabkota	pad_folder	base_name
0	JTM001	Kabupaten Bangkalan	kab_bangkalan	Bangkalan
1	JTM002	Kabupaten Banyuwangi	kab_banyuwangi	Banyuwangi
2	JTM003	Kabupaten Blitar	kab_blitar	Blitar
3	JTM004	Kabupaten Bojonegoro	kab_bojonegoro	Bojonegoro
4	JTM005	Kabupaten Bondowoso	kab_bondowoso	Bondowoso

```
def extract_pad_panel(pad_root: Path, region_ref: pd.DataFrame):
    records = []
    issues = []

    for _, region_row in region_ref.iterrows():
        region_folder = pad_root / region_row["pad_folder"]
        yearly_files = sorted(region_folder.glob("*.xls*"))

        if not yearly_files:
            issues.append(f"Tidak ada file PAD di {region_folder}")
            continue

        for file_path in yearly_files:
            tahun = extract_year(file_path.name)
            if tahun not in YEARS:
                continue
```

```
            try:
                df = pd.read_excel(file_path)
            except Exception as exc:
                issues.append(f"Gagal baca {file_path.name}: {exc}")
                continue

            if not {"Akun", "Realisasi"}.issubset(df.columns):
                issues.append(f"Kolom wajib tidak lengkap pada {file_
                continue

            akun = df["Akun"].astype(str).str.strip().str.lower()
            realisasi = df["Realisasi"].apply(clean_numeric)

            pad_mask = akun.eq("pad")
            belanja_mask = akun.str.contains(r"belanja\s*daerah", na=

            pad_val = realisasi[pad_mask].iloc[0] if pad_mask.any() e
            belanja_val = realisasi[belanja_mask].iloc[0] if belanja_

            records.append(
                {
                    "kode_kabkota": region_row["kode_kabkota"],
                    "nama_kabkota": region_row["nama_kabkota"],
                    "tahun": tahun,
                    "PAD": pad_val,
                    "total_belanja_daerah": belanja_val,
                }
            )

    pad_df = pd.DataFrame(records)
    return pad_df, issues


def infer_region_value_columns(df: pd.DataFrame, output_col: str = ""
    cols = list(df.columns)

    region_col = None
    for c in cols:
        cname = normalize_text(c)
        if "kab" in cname or "kota" in cname:
            region_col = c
            break
    if region_col is None:
        region_col = cols[0]

    value_candidates = [c for c in cols if c != region_col]
    if not value_candidates:
        return region_col, None

    prefer_tokens_map = {
        "jumlah_penduduk": ["laki-laki + perempuan", "jumlah penduduk
        "jumlah_penduduk_miskin": ["miskin", "ribu", "jiwa"],
        "IPM": ["ipm"],
        "TPT": ["tpt", "pengangguran"],
        "PDRB_per_kapita_ADHK": ["pdrb", "adhk", "kapita"],
```

```
        }
        prefer_tokens = prefer_tokens_map.get(output_col, [output_col.low

        def candidate_score(col_name: str):
            series = df[col_name]
            numeric_series = series.apply(clean_numeric)
            numeric_count = int(numeric_series.notna().sum())

            text_probe_raw = " ".join(series.fillna("").astype(str).head(
            text_probe = normalize_text(text_probe_raw)
            col_probe = normalize_text(col_name)
            probe_all = f"{col_probe} {text_probe}"

            token_hits = sum(token in probe_all for token in prefer_token

            # Prioritas kuat untuk total penduduk (laki-laki + perempuan)
            hard_bonus = 0
            if output_col == "jumlah_penduduk":
                has_male = "laki-laki" in probe_all or "laki laki" in pro
                has_female = "perempuan" in probe_all
                has_total_hint = "+" in text_probe_raw or "total" in prob
                if has_male and has_female and has_total_hint:
                    hard_bonus += 100

            return hard_bonus, token_hits, numeric_count

        ranked = sorted(value_candidates, key=candidate_score, reverse=Tr
        return region_col, ranked[0]


    def standardize_unit_to_jiwa(df: pd.DataFrame, value_col: str, output_
        if df.empty:
            return df

        out = df.copy()
        if output_col not in {"jumlah_penduduk", "jumlah_penduduk_miskin"
            return out

        for year, idx in out.groupby("tahun").groups.items():
            vals = out.loc[idx, value_col].dropna()
            if vals.empty:
                continue

            median_val = float(vals.median())
            needs_scale = False

            if output_col == "jumlah_penduduk":
                needs_scale = median_val < 10000
            elif output_col == "jumlah_penduduk_miskin":
                needs_scale = median_val < 2000

            if needs_scale:
                out.loc[idx, value_col] = out.loc[idx, value_col] * 1000
                issues.append(
                    f"Normalisasi satuan {output_col} tahun {year}: dikal
```

```
            )

    return out


def load_indicator_dir(indicator_dir: Path, output_col: str, canonica
    records = []
    issues = []

    files = sorted(indicator_dir.glob("*.xls*"))
    if not files:
        raise FileNotFoundError(f"Tidak ada file .xlsx di {indicator_

    for file_path in files:
        tahun = extract_year(file_path.name)
        if tahun not in YEARS:
            continue

        try:
            df = pd.read_excel(file_path)
        except Exception as exc:
            issues.append(f"Gagal baca {file_path.name}: {exc}")
            continue

        region_col, value_col = infer_region_value_columns(df, output_
        if value_col is None:
            issues.append(f"Gagal infer kolom nilai pada {file_path.n
            continue

        temp = pd.DataFrame(
            {
                "raw_region": df[region_col],
                "raw_value": df[value_col],
            }
        )

        temp["nama_kabkota"] = temp["raw_region"].apply(
            lambda x: harmonize_region_name(x, canonical_lookup, base_
        )
        temp[output_col] = temp["raw_value"].apply(clean_numeric)
        temp["tahun"] = tahun

        temp = temp[temp["nama_kabkota"].notna()].copy()
        temp = temp[["nama_kabkota", "tahun", output_col]]
        temp = temp.groupby(["nama_kabkota", "tahun"], as_index=False

        temp = standardize_unit_to_jiwa(temp, output_col, output_col,
        records.append(temp)

    out = pd.concat(records, ignore_index=True) if records else pd.Da
    return out, issues


# 1) Extract PAD + total belanja daerah
pad_df, pad_issues = extract_pad_panel(pad_root, region_ref)
```

```
# 2) Extract other indicators
ipm_df, ipm_issues = load_indicator_dir(ROOT / "ipm", "IPM", canonica
pop_df, pop_issues = load_indicator_dir(ROOT / "jumlah_penduduk", "ju
tpt_df, tpt_issues = load_indicator_dir(ROOT / "TPT", "TPT", canonica
poor_df, poor_issues = load_indicator_dir(ROOT / "penduduk_miskin", "
pdrb_df, pdrb_issues = load_indicator_dir(ROOT / "pdrb_perkapita", "P

all_issues = pad_issues + ipm_issues + pop_issues + tpt_issues + poor
if all_issues:
    print("Catatan parsing (sample hingga 20):")
    for msg in all_issues[:20]:
        print("-", msg)
else:
    print("Parsing selesai tanpa isu kritis.")

print("\nUkuran dataset mentah:")
print("PAD:", pad_df.shape)
print("IPM:", ipm_df.shape)
print("Jumlah Penduduk:", pop_df.shape)
print("TPT:", tpt_df.shape)
print("Penduduk Miskin:", poor_df.shape)
print("PDRB per kapita:", pdrb_df.shape)

print("\nRingkas skala setelah normalisasi:")
print(
    pop_df.groupby("tahun")["jumlah_penduduk"].median().round(2).to_s
)
print(
    poor_df.groupby("tahun")["jumlah_penduduk_miskin"].median().round
)

pad_df.head()
```

```
Catatan parsing (sample hingga 20):
- Normalisasi satuan jumlah_penduduk tahun 2024: dikali 1000 (indikasi
- Normalisasi satuan jumlah_penduduk_miskin tahun 2020: dikali 1000 (i
- Normalisasi satuan jumlah_penduduk_miskin tahun 2021: dikali 1000 (i
- Normalisasi satuan jumlah_penduduk_miskin tahun 2022: dikali 1000 (i
- Normalisasi satuan jumlah_penduduk_miskin tahun 2023: dikali 1000 (i
- Normalisasi satuan jumlah_penduduk_miskin tahun 2024: dikali 1000 (i

Ukuran dataset mentah:
PAD: (190, 5)
IPM: (190, 3)
Jumlah Penduduk: (190, 3)
TPT: (190, 3)
Penduduk Miskin: (190, 3)
PDRB per kapita: (190, 3)

Ringkas skala setelah normalisasi:
tahun
2020   1,075,076.0000
2021   1,084,150.0000
2022   1,095,978.5000
2023   1,107,764.5000
2024   1,108,200.0000
tahun
2020   120,765.0000
2021   123,035.0000
2022   112,330.0000
2023   114,360.0000
2024   107,490.0000
```

	kode_kabkota	nama_kabkota	tahun	PAD	total_belanja_c
0	JTM001	Kabupaten Bangkalan	2020	233,177,792,497.4900	2,116,224,183,51
1	JTM001	Kabupaten Bangkalan	2021	271,656,642,591.0000	2,211,221,771,30
2	JTM001	Kabupaten Bangkalan	2022	281,897,411,487.8000	2,264,589,338,63
3	JTM001	Kabupaten Bangkalan	2023	353,692,219,635.3800	2,042,554,668,83
4	JTM001	Kabupaten Bangkalan	2024	414,115,920,457.9500	2,459,309,778,12

```
# Merge into single panel
panel = pad_df.copy()
for extra_df in [ipm_df, pdrb_df, tpt_df, pop_df, poor_df]:
    panel = panel.merge(extra_df, on=["nama_kabkota", "tahun"], how="

# Ensure exact required columns and ordering
required_cols = [
    "kode_kabkota",
    "nama_kabkota",
    "tahun",
```

```
    "PAD",
    "total_belanja_daerah",
    "IPM",
    "PDRB_per_kapita_ADHK",
    "TPT",
    "jumlah_penduduk",
    "jumlah_penduduk_miskin",
]
panel = panel[required_cols].copy()

# Derived variables
panel["PAD_per_capita"] = panel["PAD"] / panel["jumlah_penduduk"]
panel["belanja_per_capita"] = panel["total_belanja_daerah"] / panel["
panel["poor_rate_1000"] = panel["jumlah_penduduk_miskin"] / panel["ju

# Sort panel
panel = panel.sort_values(["kode_kabkota", "tahun"]).reset_index(drop

# One-year lags by kabupaten/kota
lag_map = {
    "poor_rate_1000": "poor_rate_1000_lag1",
    "IPM": "IPM_lag1",
    "TPT": "TPT_lag1",
    "PAD_per_capita": "PAD_per_capita_lag1",
    "belanja_per_capita": "belanja_per_capita_lag1",
    "PDRB_per_kapita_ADHK": "PDRB_per_kapita_ADHK_lag1",
}
for src_col, lag_col in lag_map.items():
    panel[lag_col] = panel.groupby("kode_kabkota")[src_col].shift(1)

# Drop only rows missing because of lag construction
lag_cols = list(lag_map.values())
panel_lagged = panel.dropna(subset=lag_cols).copy()

# Basic diagnostics
print("Panel full shape:", panel.shape)
print("Panel lagged shape:", panel_lagged.shape)
print("Jumlah kab/kota unik:", panel_lagged["kode_kabkota"].nunique()
print("Rentang tahun lagged:", panel_lagged["tahun"].min(), "-", pane

missing_share = panel[required_cols].isna().mean().sort_values(ascend
print("\nProporsi missing per variabel inti:")
print((missing_share * 100).round(2).astype(str) + "%")

panel_lagged.head()
```

```
Panel full shape: (190, 19)
Panel lagged shape: (152, 19)
Jumlah kab/kota unik: 38
Rentang tahun lagged: 2021 - 2024

Proporsi missing per variabel inti:
kode_kabkota              0.0%
nama_kabkota              0.0%
tahun                     0.0%
PAD                       0.0%
total_belanja_daerah      0.0%
IPM                       0.0%
PDRB_per_kapita_ADHK      0.0%
TPT                       0.0%
jumlah_penduduk           0.0%
jumlah_penduduk_miskin    0.0%
dtype: str
```

	kode_kabkota	nama_kabkota	tahun	PAD	total_belanja_d
1	JTM001	Kabupaten Bangkalan	2021	271,656,642,591.0000	2,211,221,771,30
2	JTM001	Kabupaten Bangkalan	2022	281,897,411,487.8000	2,264,589,338,63
3	JTM001	Kabupaten Bangkalan	2023	353,692,219,635.3800	2,042,554,668,83
4	JTM001	Kabupaten Bangkalan	2024	414,115,920,457.9500	2,459,309,778,12
6	JTM002	Kabupaten Banyuwangi	2021	520,021,954,381.2000	3,093,593,254,51

Strategi Pemodelan Panel untuk Simulasi Kebijakan

Kita membangun 3 model supervised terpisah dengan regularisasi Ridge:

- **Model 1**: target `poor_rate_1000`
- **Model 2**: target `IPM`
- **Model 3**: target `TPT`

Validasi menggunakan **GroupKFold** dengan `kode_kabkota` sebagai grup agar evaluasi tidak mencampur pola antar-wilayah secara tidak tepat.

> Penting: model ini digunakan untuk simulasi skenario kebijakan dan analisis sensitivitas kebijakan fiskal, bukan untuk klaim kausal murni.

```
def train_panel_ridge(df: pd.DataFrame, target_col: str, target_lag_c
    feature_cols = [
        target_lag_col,
        "PAD_per_capita_lag1",
```

```
        "belanja_per_capita_lag1",
        "PDRB_per_kapita_ADHK_lag1",
    ]

    train_df = df.dropna(subset=feature_cols + [target_col, group_col
    X = train_df[feature_cols]
    y = train_df[target_col]
    groups = train_df[group_col]

    n_groups = groups.nunique()
    if n_groups < 2:
        raise ValueError(f"Jumlah group terlalu sedikit untuk CV: {n_

    n_splits = min(5, n_groups)
    gkf = GroupKFold(n_splits=n_splits)

    rmse_list = []
    mae_list = []

    for fold, (tr_idx, te_idx) in enumerate(gkf.split(X, y, groups=gr
        pipe = Pipeline(
            steps=[
                ("scaler", StandardScaler()),
                ("model", Ridge(alpha=8.0)),
            ]
        )
        pipe.fit(X.iloc[tr_idx], y.iloc[tr_idx])
        y_pred = pipe.predict(X.iloc[te_idx])

        rmse = mean_squared_error(y.iloc[te_idx], y_pred) ** 0.5
        mae = mean_absolute_error(y.iloc[te_idx], y_pred)
        rmse_list.append(rmse)
        mae_list.append(mae)

    # Fit final model on full lagged sample
    final_pipe = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
            ("model", Ridge(alpha=8.0)),
        ]
    )
    final_pipe.fit(X, y)

    metrics = {
        "target": target_col,
        "n_obs": len(train_df),
        "n_groups": n_groups,
        "cv_rmse_mean": float(np.mean(rmse_list)),
        "cv_rmse_std": float(np.std(rmse_list)),
        "cv_mae_mean": float(np.mean(mae_list)),
        "cv_mae_std": float(np.std(mae_list)),
    }

    return {
        "model": final_pipe,
```

```
        "feature_cols": feature_cols,
        "metrics": metrics,
    }


# Train 3 models (same base fiscal-economic lag features + masing-mas
model_specs = {
    "poor_rate": {"target": "poor_rate_1000", "lag": "poor_rate_1000_
    "IPM": {"target": "IPM", "lag": "IPM_lag1"},
    "TPT": {"target": "TPT", "lag": "TPT_lag1"},
}

fitted_models = {}
metrics_rows = []

for model_name, spec in model_specs.items():
    result = train_panel_ridge(panel_lagged, target_col=spec["target"
    fitted_models[model_name] = result
    row = {"model": model_name, **result["metrics"]}
    metrics_rows.append(row)

metrics_df = pd.DataFrame(metrics_rows)
print("Hasil evaluasi GroupKFold (Ridge):")
metrics_df

# Baseline prediction for the last available year
last_year = int(panel_lagged["tahun"].max())
baseline_input = panel_lagged[panel_lagged["tahun"] == last_year].cop

if baseline_input.empty:
    raise ValueError("Tidak ada data untuk baseline pada tahun terakh

baseline = baseline_input[["kode_kabkota", "nama_kabkota"]].copy()
baseline["poor_rate_baseline"] = fitted_models["poor_rate"]["model"].
    baseline_input[fitted_models["poor_rate"]["feature_cols"]]
)
baseline["IPM_baseline"] = fitted_models["IPM"]["model"].predict(
    baseline_input[fitted_models["IPM"]["feature_cols"]]
)
baseline["TPT_baseline"] = fitted_models["TPT"]["model"].predict(
    baseline_input[fitted_models["TPT"]["feature_cols"]]
)

print(f"Baseline disusun untuk tahun: {last_year}, jumlah wilayah: {l
baseline.head()
```

```
Hasil evaluasi GroupKFold (Ridge):
Baseline disusun untuk tahun: 2024, jumlah wilayah: 38
```

	kode_kabkota	nama_kabkota	poor_rate_baseline	IPM_baseline	TPT_b
4	JTM001	Kabupaten Bangkalan	308.7608	67.9617	
9	JTM002	Kabupaten Banyuwangi	108.5484	74.1675	
14	JTM003	Kabupaten Blitar	132.2195	73.3191	
19	JTM004	Kabupaten Bojonegoro	189.7035	72.7936	
24	JTM005	Kabupaten Bondowoso	218.5772	71.3350	

```
def run_scenario_prediction(base_input: pd.DataFrame, fitted_models: 
    """
    multipliers example:
    {
        "belanja_per_capita_lag1": 0.10,
        "PDRB_per_kapita_ADHK_lag1": 0.05,
    }
    """
    scenario_input = base_input.copy()

    for col, pct in multipliers.items():
        if col not in scenario_input.columns:
            raise KeyError(f"Kolom skenario tidak ditemukan: {col}")
        scenario_input[col] = scenario_input[col] * (1 + pct)

    out = pd.DataFrame(
        {
            "kode_kabkota": scenario_input["kode_kabkota"],
            "nama_kabkota": scenario_input["nama_kabkota"],
            f"poor_rate_{scenario_label}": fitted_models["poor_rate"]
                scenario_input[fitted_models["poor_rate"]["feature_co
            ),
            f"IPM_{scenario_label}": fitted_models["IPM"]["model"].pr
                scenario_input[fitted_models["IPM"]["feature_cols"]]
            ),
            f"TPT_{scenario_label}": fitted_models["TPT"]["model"].pr
                scenario_input[fitted_models["TPT"]["feature_cols"]]
            ),
        }
    )
    return out


# Scenario definitions
scenarios = {
```

```
    "s1": {"belanja_per_capita_lag1": 0.10},
    "s2": {"PDRB_per_kapita_ADHK_lag1": 0.05},
    "s3": {
        "belanja_per_capita_lag1": 0.10,
        "PAD_per_capita_lag1": 0.10,
        "PDRB_per_kapita_ADHK_lag1": 0.05,
    },
}

output = baseline.copy()

for s_label, multiplier_dict in scenarios.items():
    s_pred = run_scenario_prediction(
        base_input=baseline_input,
        fitted_models=fitted_models,
        multipliers=multiplier_dict,
        scenario_label=s_label,
    )

    output = output.merge(s_pred, on=["kode_kabkota", "nama_kabkota"]

    output[f"delta_poor_rate_{s_label}"] = output[f"poor_rate_{s_labe
    output[f"delta_IPM_{s_label}"] = output[f"IPM_{s_label}"] - outpu
    output[f"delta_TPT_{s_label}"] = output[f"TPT_{s_label}"] - outpu

# Final tidy column order
final_cols = [
    "kode_kabkota",
    "nama_kabkota",
    "poor_rate_baseline",
    "IPM_baseline",
    "TPT_baseline",
    "poor_rate_s1",
    "IPM_s1",
    "TPT_s1",
    "poor_rate_s2",
    "IPM_s2",
    "TPT_s2",
    "poor_rate_s3",
    "IPM_s3",
    "TPT_s3",
    "delta_poor_rate_s1",
    "delta_IPM_s1",
    "delta_TPT_s1",
    "delta_poor_rate_s2",
    "delta_IPM_s2",
    "delta_TPT_s2",
    "delta_poor_rate_s3",
    "delta_IPM_s3",
    "delta_TPT_s3",
]

output = output[final_cols].sort_values("kode_kabkota").reset_index(d

save_path = ROOT / "policy_simulation_output.csv"
```

```
output.to_csv(save_path, index=False)

print(f"File output berhasil disimpan: {save_path}")
output.head(10)
```

```
File output berhasil disimpan: D:\Downloads\infografis3\infografis3\po
```

	kode_kabkota	nama_kabkota	poor_rate_baseline	IPM_baseline	TPT_ba
0	JTM001	Kabupaten Bangkalan	33.7828	67.9617	
1	JTM002	Kabupaten Banyuwangi	29.2562	74.1675	
2	JTM003	Kabupaten Blitar	28.7307	73.3191	
3	JTM004	Kabupaten Bojonegoro	31.2824	72.7936	
4	JTM005	Kabupaten Bondowoso	40.4773	71.3350	
5	JTM006	Kabupaten Gresik	27.8332	78.7364	
6	JTM007	Kabupaten Jember	31.5418	71.1726	
7	JTM008	Kabupaten Jombang	29.6525	75.4943	
8	JTM009	Kabupaten Kediri	31.5301	74.9576	
9	JTM010	Kabupaten Lamongan	32.8002	75.5039	

Visualisasi Hasil Simulasi Skenario

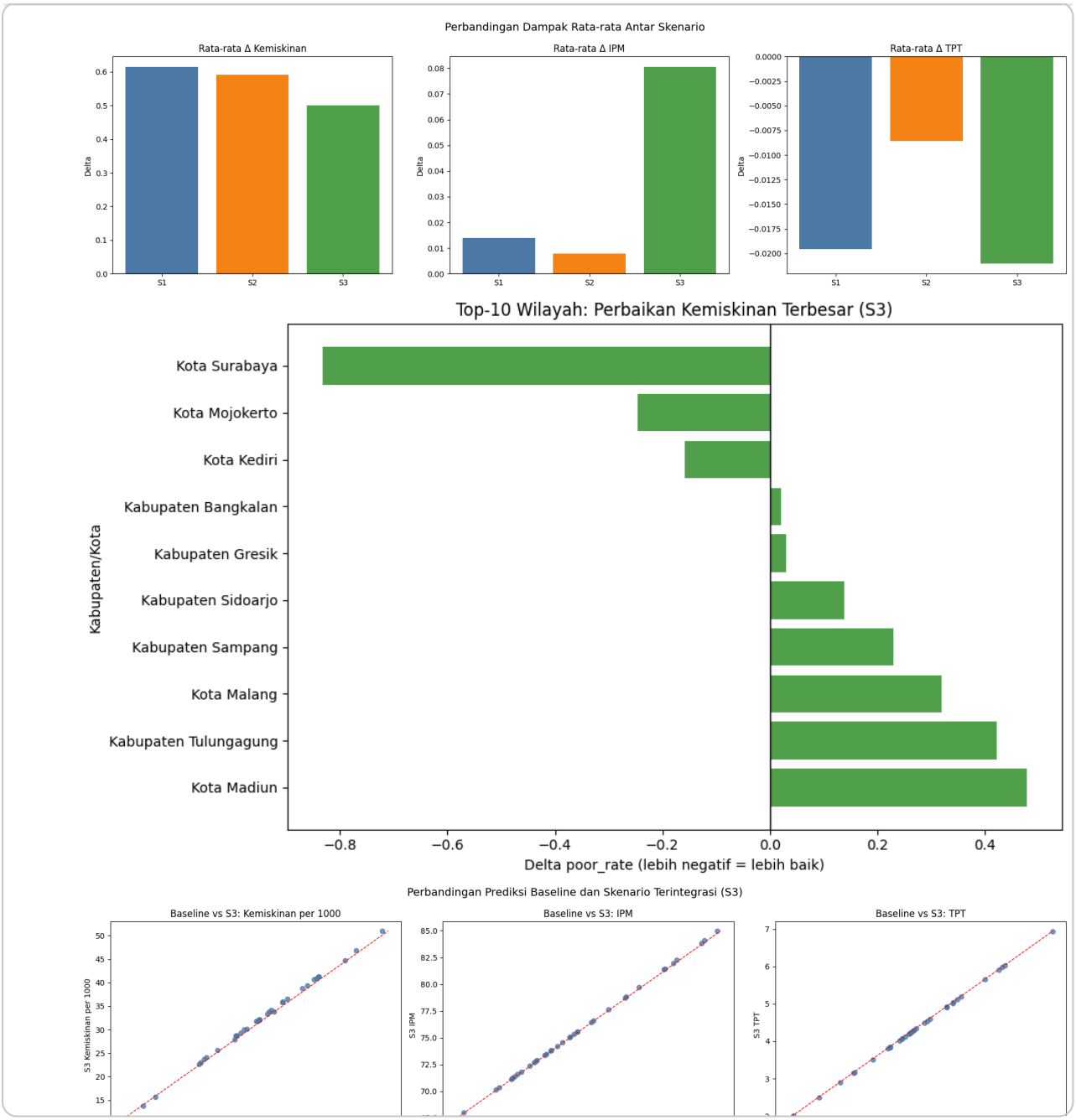
Bagian ini menampilkan ringkasan visual dari `policy_simulation_output.csv` / `output`:

1. Rata-rata perubahan indikator (`delta`) per skenario.
2. Top-10 wilayah dengan perbaikan kemiskinan terbesar pada S3.
3. Perbandingan baseline vs skenario terintegrasi (S3).

```
import matplotlib.pyplot as plt

# Gunakan output dari cell simulasi. Jika belum ada, coba baca dari C
if "output" not in globals():
    output = pd.read_csv(ROOT / "policy_simulation_output.csv")

# ==============================
```

```
# 1) Rata-rata delta per skenario
# ==============================
summary_delta = pd.DataFrame(
    {
        "S1": [
            output["delta_poor_rate_s1"].mean(),
            output["delta_IPM_s1"].mean(),
            output["delta_TPT_s1"].mean(),
        ],
        "S2": [
            output["delta_poor_rate_s2"].mean(),
            output["delta_IPM_s2"].mean(),
            output["delta_TPT_s2"].mean(),
        ],
        "S3": [
            output["delta_poor_rate_s3"].mean(),
            output["delta_IPM_s3"].mean(),
            output["delta_TPT_s3"].mean(),
        ],
    },
    index=["Kemiskinan (per 1000)", "IPM", "TPT"],
)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Kemiskinan: semakin negatif semakin baik
axes[0].bar(["S1", "S2", "S3"], summary_delta.loc["Kemiskinan (per 10
axes[0].axhline(0, color="black", linewidth=1)
axes[0].set_title("Rata-rata Δ Kemiskinan")
axes[0].set_ylabel("Delta")

# IPM: semakin positif semakin baik
axes[1].bar(["S1", "S2", "S3"], summary_delta.loc["IPM", ["S1", "S2",
axes[1].axhline(0, color="black", linewidth=1)
axes[1].set_title("Rata-rata Δ IPM")
axes[1].set_ylabel("Delta")

# TPT: semakin negatif semakin baik
axes[2].bar(["S1", "S2", "S3"], summary_delta.loc["TPT", ["S1", "S2",
axes[2].axhline(0, color="black", linewidth=1)
axes[2].set_title("Rata-rata Δ TPT")
axes[2].set_ylabel("Delta")

plt.suptitle("Perbandingan Dampak Rata-rata Antar Skenario", fontsize=
plt.tight_layout()
plt.show()

# ========================================
# 2) Top-10 perbaikan kemiskinan terbesar S3
# ========================================
top10_s3_poor = output.sort_values("delta_poor_rate_s3").head(10).cop

plt.figure(figsize=(10, 6))
plt.barh(top10_s3_poor["nama_kabkota"], top10_s3_poor["delta_poor_rat
plt.axvline(0, color="black", linewidth=1)
```

```
plt.title("Top-10 Wilayah: Perbaikan Kemiskinan Terbesar (S3)")
plt.xlabel("Delta poor_rate (lebih negatif = lebih baik)")
plt.ylabel("Kabupaten/Kota")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# ======================================
# 3) Baseline vs S3 untuk tiga indikator
# ======================================
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

pairs = [
    ("poor_rate_baseline", "poor_rate_s3", "Kemiskinan per 1000"),
    ("IPM_baseline", "IPM_s3", "IPM"),
    ("TPT_baseline", "TPT_s3", "TPT"),
]

for ax, (xcol, ycol, label) in zip(axes, pairs):
    ax.scatter(output[xcol], output[ycol], alpha=0.75, color="#4C78A8
    mn = min(output[xcol].min(), output[ycol].min())
    mx = max(output[xcol].max(), output[ycol].max())
    ax.plot([mn, mx], [mn, mx], linestyle="--", color="red", linewidt
    ax.set_xlabel(f"Baseline {label}")
    ax.set_ylabel(f"S3 {label}")
    ax.set_title(f"Baseline vs S3: {label}")

plt.suptitle("Perbandingan Prediksi Baseline dan Skenario Terintegras
plt.tight_layout()
plt.show()

summary_delta
```



Visualisasi Spasial (Choropleth)

Peta berikut menampilkan sebaran dampak skenario terintegrasi **S3** pada indikator kemiskinan (`delta_poor_rate_s3`) per kabupaten/kota di Jawa Timur.

Interpretasi warna:

- Nilai **lebih negatif** (`delta_poor_rate_s3 < 0`) berarti perbaikan kemiskinan lebih kuat.
- Nilai **lebih positif** berarti perbaikan lebih lemah atau potensi memburuk.

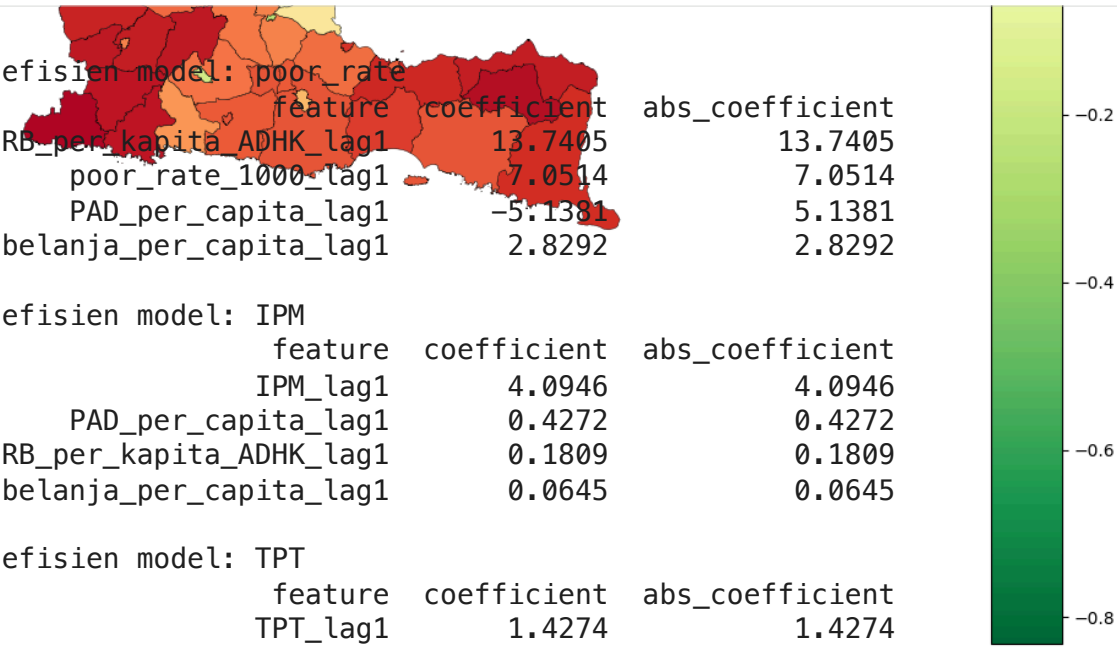
```
# Choropleth dampak kemiskinan skenario S3 (robust join key)
path_geojson = ROOT / "Kabupaten-Kota (Provinsi Jawa Timur).geojson"

if "output" not in globals():
    output = pd.read_csv(ROOT / "policy_simulation_output.csv")
```

```
def _pick_name_column(gdf):
    """Pilih kolom nama kab/kota paling mungkin dari GeoJSON."""
    cols = [c for c in gdf.columns if c != "geometry"]
    if not cols:
        return None

    lowered = {c: str(c).strip().lower() for c in cols}
    n = len(gdf)

    # Kandidat string dengan variasi cukup (hindari kolom konstan sep
    candidates = []
    for c in cols:
        series = gdf[c]
        nunique = series.nunique(dropna=True)
        ratio = nunique / max(n, 1)
        is_text_like = series.dtype == object
        if is_text_like and nunique >= 10:
            candidates.append((c, lowered[c], nunique, ratio))

    if not candidates:
        return None

    # Skor berbasis nama kolom + jumlah unique
    def score(item):
        c, lc, nunique, ratio = item
        s = 0

        # Prioritas tinggi untuk pola lazim level kab/kota
        if lc in {"name_2", "name2", "nama_kabkota", "kabkot", "kabup
            s += 100
        if "name_2" in lc or "name2" in lc:
            s += 50
        if any(t in lc for t in ["kab", "kota", "regency", "city", "d
            s += 30
        if any(t in lc for t in ["nama", "name"]):
            s += 15

        # Prefer kolom yang hampir unik per fitur
        s += int(ratio * 20)
        s += min(nunique, 50)
        return s

    candidates_sorted = sorted(candidates, key=score, reverse=True)
    return candidates_sorted[0][0]


def _clean_region_text(text):
    s = normalize_text(text)
    s = re.sub(r"^kab\.?\s+", "kabupaten ", s)
    s = re.sub(r"^kota\.?\s+", "kota ", s)
    s = re.sub(r"^kabupaten\s+", "kabupaten ", s)
    s = re.sub(r"\s+", " ", s).strip()
    return s
```

```
def _to_canonical_region_name(text):
    cleaned = _clean_region_text(text)
    mapped = harmonize_region_name(cleaned, canonical_lookup, base_lo
    if pd.isna(mapped):
        # fallback: coba tanpa prefix kab/kota
        base = re.sub(r"^(kabupaten|kota)\s+", "", cleaned)
        mapped = harmonize_region_name(base, canonical_lookup, base_l
    return mapped


try:
    import geopandas as gpd

    if not path_geojson.exists():
        print(f"GeoJSON tidak ditemukan: {path_geojson}")
    else:
        gdf = gpd.read_file(path_geojson)
        output_plot = output.copy()

        # --- Opsi 1: join by kode_kabkota jika tersedia ---
        if "kode_kabkota" in gdf.columns:
            gdf["kode_kabkota"] = gdf["kode_kabkota"].astype(str)
            output_plot["kode_kabkota"] = output_plot["kode_kabkota"]

            gdf_plot = gdf.merge(
                output_plot[["kode_kabkota", "nama_kabkota", "delta_p
                on="kode_kabkota",
                how="left",
            )
            join_note = "Join key: kode_kabkota"

        # --- Opsi 2: fallback join by nama wilayah ---
        else:
            name_col = _pick_name_column(gdf)
            if name_col is None:
                raise KeyError(
                    "GeoJSON tidak memiliki kolom 'kode_kabkota' maup
                )

            gdf_plot = gdf.copy()
            gdf_plot["nama_kabkota_geo"] = gdf_plot[name_col].astype(
            gdf_plot["nama_kabkota_join"] = gdf_plot["nama_kabkota_ge

            gdf_plot["join_key"] = gdf_plot["nama_kabkota_join"].appl
            output_plot["join_key"] = output_plot["nama_kabkota"].app

            gdf_plot = gdf_plot.merge(
                output_plot[["join_key", "nama_kabkota", "delta_poor_
                on="join_key",
                how="left",
            )
            join_note = f"Join key fallback: nama wilayah (GeoJSON: '
```

```
        fig, ax = plt.subplots(1, 1, figsize=(10, 12))
        gdf_plot.plot(
            column="delta_poor_rate_s3",
            cmap="RdYlGn_r",  # negatif (lebih baik) cenderung hijau
            linewidth=0.3,
            edgecolor="black",
            legend=True,
            missing_kwds={
                "color": "lightgrey",
                "label": "Data tidak tersedia",
            },
            ax=ax,
        )

        ax.set_title("Choropleth Dampak S3 pada Kemiskinan (delta_poo
        ax.set_axis_off()
        plt.tight_layout()
        plt.show()

        print(join_note)
        matched = gdf_plot["delta_poor_rate_s3"].notna().sum()
        total = len(gdf_plot)
        print(f"Wilayah ter-join: {matched}/{total}")

        # Debug singkat jika masih banyak tidak match
        if matched < total:
            missing_rows = gdf_plot[gdf_plot["delta_poor_rate_s3"].isı
            if "nama_kabkota_geo" in missing_rows.columns:
                print("Contoh nama GeoJSON yang belum match:")
                print(missing_rows["nama_kabkota_geo"].dropna().head(

        # Tabel ringkas top perbaikan (delta paling negatif)
        show_cols = [c for c in ["kode_kabkota", "nama_kabkota", "nam
        display(gdf_plot[show_cols].sort_values("delta_poor_rate_s3")

except ImportError:
    print("Package 'geopandas' belum terpasang. Jalankan: pip install
```

```
# Coefficients / feature importance (Ridge coefficients)
coef_tables = {}

for model_name, fitted in fitted_models.items():
    feature_cols = fitted["feature_cols"]
    coef = fitted["model"].named_steps["model"].coef_

    coef_df = pd.DataFrame(
        {
            "feature": feature_cols,
            "coefficient": coef,
            "abs_coefficient": np.abs(coef),
        }
    ).sort_values("abs_coefficient", ascending=False)

    coef_tables[model_name] = coef_df
    print(f"\nKoefisien model: {model_name}")
    print(coef_df.to_string(index=False))

coef_tables["poor_rate"]
```




```
Koefisien model: poor_rate
                  feature  coefficient  abs_coefficient
PDRB_per_kapita_ADHK_lag1      13.7405          13.7405
      poor_rate_1000_lag1       7.0514           7.0514
      PAD_per_capita_lag1      -5.1381           5.1381
  belanja_per_capita_lag1       2.8292           2.8292

Koefisien model: IPM
                  feature  coefficient  abs_coefficient
                 IPM_lag1       4.0946           4.0946
      PAD_per_capita_lag1       0.4272           0.4272
PDRB_per_kapita_ADHK_lag1       0.1809           0.1809
  belanja_per_capita_lag1       0.0645           0.0645

Koefisien model: TPT
                  feature  coefficient  abs_coefficient
                 TPT_lag1       1.4274           1.4274
PDRB_per_kapita_ADHK_lag1      -0.1997           0.1997
  belanja_per_capita_lag1      -0.0899           0.0899
      PAD_per_capita_lag1       0.0518           0.0518
Join-key-fallback: nama_wilayah (GeoJSON: 'NAME_2')
Wilayah ter-join: 38/38
```

	nama_kabkota	nama_kabkota_join	delta_poor_rate_s3
	feature	coefficient	abs_coefficient
3	PDRB_per_kapita_ADHK_lag1	13.7405	13.7405	-0.8316
14	Kota Mojokerto	Kota Mojokerto	-0.2453
0	poor_rate_1000_lag1	7.0514	7.0514	
1	PAD_per_capita_lag1	-5.1381	5.1381	-0.1585
0	Kabupaten Bangkalan	Kabupaten Bangkalan	0.0207
2	belanja_per_capita_lag1	2.8292	2.8292	
6	Kabupaten Gresik	Kabupaten Gresik	0.0306
31	Kabupaten Sidoarjo	Kabupaten Sidoarjo	0.1379
30	Kabupaten Sampang	Kabupaten Sampang	0.2300
13	Kota Malang	Kota Malang	0.3197

Panduan Interpretasi Skenario

Cara membaca hasil simulasi

- `poor_rate_baseline`, `IPM_baseline`, `TPT_baseline` adalah prediksi kondisi baseline (tahun terakhir dengan lag teramati).
- `poor_rate_sX`, `IPM_sX`, `TPT_sX` adalah prediksi pada skenario `sX`.
- `delta_*_sX = nilai_skenario - nilai_baseline`.

Interpretasi tanda delta:

- `delta_poor_rate_sX < 0` → indikasi perbaikan kemiskinan (jumlah miskin per 1000 turun).
- `delta_IPM_sX > 0` → indikasi peningkatan pembangunan manusia.
- `delta_TPT_sX < 0` → indikasi perbaikan pasar kerja (pengangguran turun).

Definisi skenario kebijakan

1. **S1 (Fiscal expansion)**: `belanja_per_capita_lag1` naik 10%.
2. **S2 (Economic improvement)**: `PDRB_per_kapita_ADHK_lag1` naik 5%.
3. **S3 (Integrated policy)**: `belanja_per_capita_lag1` naik 10%, `PAD_per_capita_lag1` naik 10%, dan `PDRB_per_kapita_ADHK_lag1` naik 5%.

Integrasi dengan GeoJSON Jawa Timur

- Gunakan `policy_simulation_output.csv` sebagai tabel atribut.
- Join ke GeoJSON kabupaten/kota Jawa Timur melalui kunci `kode_kabkota`.
- Setelah join, tiap indikator baseline/skenario/delta dapat divisualisasikan sebagai choropleth map.

```
# (Opsional) Contoh join cepat dengan GeoJSON jika file geojson terse
# Gantilah path_geojson sesuai lokasi file GeoJSON Anda.
# import geopandas as gpd

# path_geojson = ROOT / "Kabupaten-Kota (Provinsi Jawa Timur).geojson"
# if path_geojson.exists():
#     gdf = gpd.read_file(path_geojson)
#     sim = pd.read_csv(ROOT / "policy_simulation_output.csv")
#     gdf_join = gdf.merge(sim, on="kode_kabkota", how="left")
#     display(gdf_join.head())
# else:
#     print("GeoJSON tidak ditemukan. Lewati langkah join spasial.")

print("Notebook siap: ekstraksi panel, modeling, simulasi skenario, o
```

```
Notebook siap: ekstraksi panel, modeling, simulasi skenario, output CS
```

Paket Visualisasi Lengkap (Semua Perspektif)

Section ini menambahkan seluruh visualisasi analitis utama:

1. Tren time series indikator utama.
2. Distribusi indikator per tahun.
3. Choropleth multipanel (delta S3 untuk 3 indikator).
4. Ranking Top/Bottom dampak skenario.
5. Quadrant plot (trade-off dan win-win).
6. Baseline vs S1/S2/S3 (garis 45 derajat).
7. Waterfall sederhana kontribusi skenario per wilayah.
8. Heatmap korelasi fitur lag dan target.
9. Koefisien Ridge per model.
10. Kinerja model (RMSE/MAE).

> Catatan: beberapa chart membutuhkan objek di memori (`panel`, `panel_lagged`, `metrics_df`, `coef_tables`, `output`) dari cell sebelumnya.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

if "ROOT" not in globals():
    ROOT = Path(".").resolve()

# ========================================
# Guard: ambil output simulasi jika belum ada
# ========================================
if "output" not in globals():
    output = pd.read_csv(ROOT / "policy_simulation_output.csv")

# ----------------------------------------
# 1) Tren time series indikator utama
# ----------------------------------------
if "panel" in globals() and not panel.empty:
    ts_cols = ["poor_rate_1000", "IPM", "TPT", "PAD_per_capita", "bel
    ts_df = panel.groupby("tahun", as_index=False)[ts_cols].mean(nume

    fig, axes = plt.subplots(2, 3, figsize=(18, 9))
    for ax, col in zip(axes.ravel(), ts_cols):
        ax.plot(ts_df["tahun"], ts_df[col], marker="o")
        ax.set_title(f"Tren Rata-rata {col}")
        ax.set_xlabel("Tahun")
        ax.grid(alpha=0.3)
```

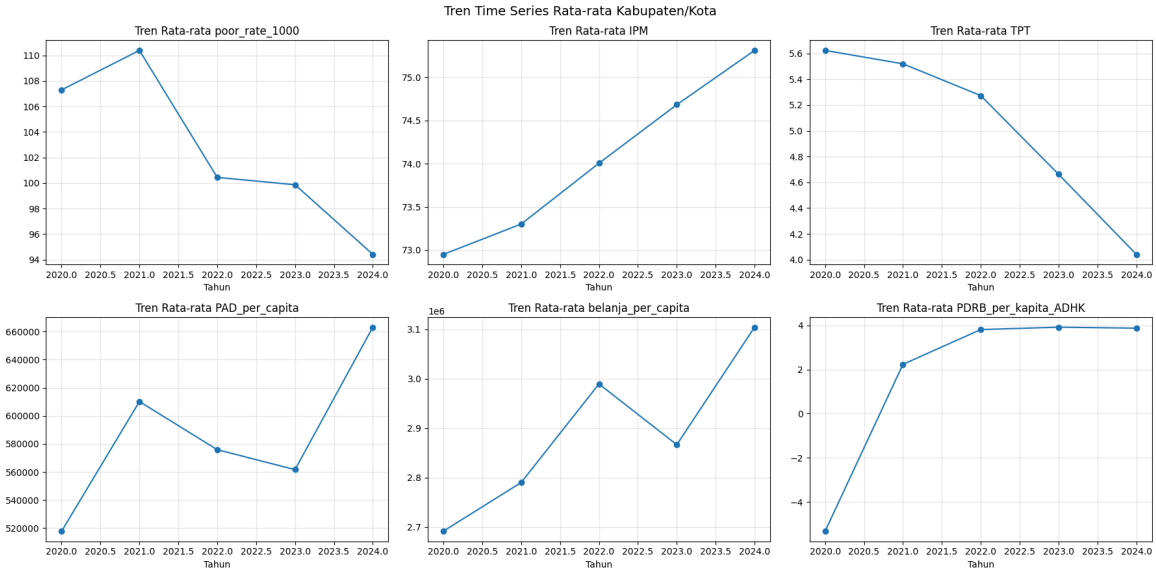
```
    plt.suptitle("Tren Time Series Rata-rata Kabupaten/Kota", fontsiz
    plt.tight_layout()
    plt.show()
else:
    print("Lewati tren time series: variabel 'panel' belum tersedia."

# ----------------------------------------
# 2) Distribusi indikator per tahun
# ----------------------------------------
if "panel" in globals() and not panel.empty:
    dist_cols = ["poor_rate_1000", "IPM", "TPT"]
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))

    years_sorted = sorted(panel["tahun"].dropna().unique())
    for ax, col in zip(axes, dist_cols):
        data_by_year = [panel.loc[panel["tahun"] == y, col].dropna().
        ax.boxplot(data_by_year, tick_labels=[str(y) for y in years_s
        ax.set_title(f"Distribusi {col} per Tahun")
        ax.set_xlabel("Tahun")
        ax.grid(axis="y", alpha=0.25)

    plt.suptitle("Pergeseran Distribusi Antar Tahun", fontsize=14)
    plt.tight_layout()
    plt.show()
else:
    print("Lewati distribusi: variabel 'panel' belum tersedia.")

# ----------------------------------------
# 3) Choropleth multipanel delta S3
# ----------------------------------------
path_geojson = ROOT / "Kabupaten-Kota (Provinsi Jawa Timur).geojson"

if path_geojson.exists():
    try:
        import geopandas as gpd

        gdf = gpd.read_file(path_geojson)
        merge_cols = ["kode_kabkota", "nama_kabkota", "delta_poor_rat

        if "kode_kabkota" in gdf.columns:
            gdf_plot = gdf.merge(output[merge_cols], on="kode_kabkota
        else:
            gdf_plot = gdf.copy()
            if "nama_kabkota" in gdf_plot.columns:
                gdf_plot["join_name"] = gdf_plot["nama_kabkota"].asty
            else:
                possible = [c for c in gdf_plot.columns if c != "geom
                pick = possible[0]
                gdf_plot["join_name"] = gdf_plot[pick].astype(str).st

            out_tmp = output.copy()
            out_tmp["join_name"] = out_tmp["nama_kabkota"].astype(str
            gdf_plot = gdf_plot.merge(out_tmp[["join_name", "delta_po

        map_specs = [
```

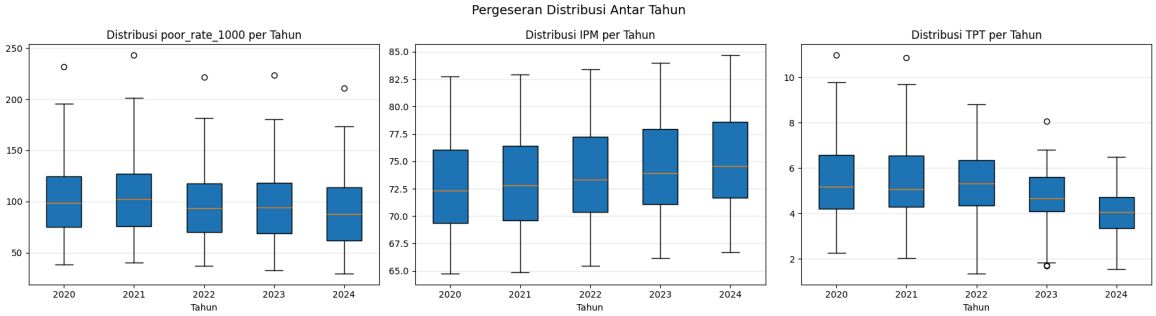
```
            ("delta_poor_rate_s3", "Δ Kemiskinan S3", "RdYlGn_r"),
            ("delta_IPM_s3", "Δ IPM S3", "RdYlGn"),
            ("delta_TPT_s3", "Δ TPT S3", "RdYlGn_r"),
        ]

        fig, axes = plt.subplots(1, 3, figsize=(20, 8))
        for ax, (col, title, cmap) in zip(axes, map_specs):
            gdf_plot.plot(
                column=col,
                cmap=cmap,
                linewidth=0.2,
                edgecolor="black",
                legend=True,
                missing_kwds={"color": "lightgrey", "label": "Data NA
                ax=ax,
            )
            ax.set_title(title)
            ax.set_axis_off()

        plt.suptitle("Choropleth Multipanel Dampak S3", fontsize=14)
        plt.tight_layout()
        plt.show()

    except ImportError:
        print("Lewati choropleth multipanel: geopandas belum terpasan
else:
    print(f"Lewati choropleth multipanel: GeoJSON tidak ditemukan di

# ----------------------------------------
# 4) Ranking Top/Bottom dampak skenario
# ----------------------------------------
def plot_top_bottom(df, col, better="lower", title_prefix=""):
    if better == "lower":
        top = df.sort_values(col).head(10)
        bottom = df.sort_values(col, ascending=False).head(10)
    else:
        top = df.sort_values(col, ascending=False).head(10)
        bottom = df.sort_values(col).head(10)

    fig, axes = plt.subplots(1, 2, figsize=(16, 5))
    axes[0].barh(top["nama_kabkota"], top[col], color="#2ca02c")
    axes[0].set_title(f"Top-10 ({title_prefix})")
    axes[0].invert_yaxis()

    axes[1].barh(bottom["nama_kabkota"], bottom[col], color="#d62728"
    axes[1].set_title(f"Bottom-10 ({title_prefix})")
    axes[1].invert_yaxis()

    for ax in axes:
        ax.axvline(0, color="black", linewidth=1)
        ax.grid(axis="x", alpha=0.2)

    plt.tight_layout()
    plt.show()
```

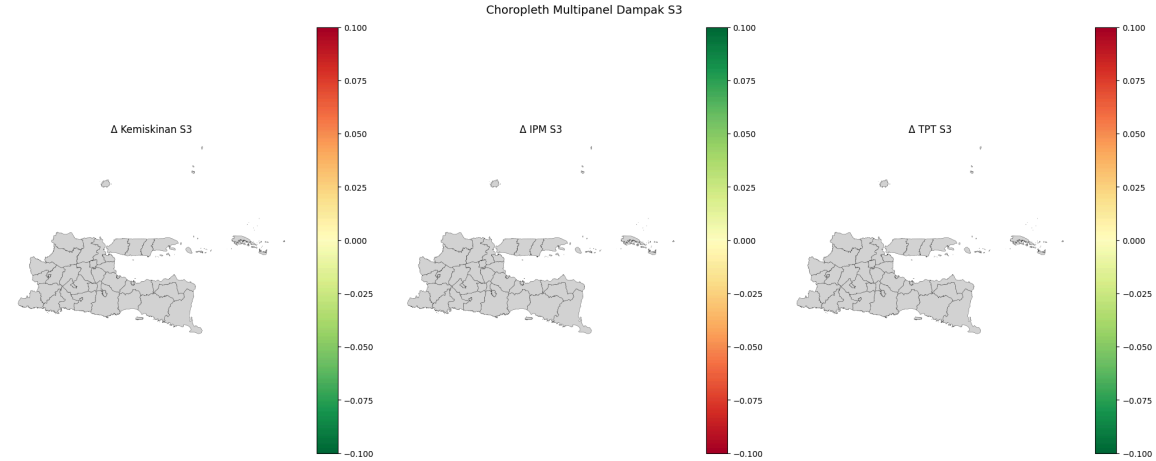
```
plot_top_bottom(output, "delta_poor_rate_s3", better="lower", title_p
plot_top_bottom(output, "delta_IPM_s3", better="higher", title_prefix=
plot_top_bottom(output, "delta_TPT_s3", better="lower", title_prefix='

# ----------------------------------------
# 5) Quadrant plot (IPM vs Kemiskinan)
# ----------------------------------------
plt.figure(figsize=(9, 7))
plt.scatter(output["delta_poor_rate_s3"], output["delta_IPM_s3"], alpl
plt.axhline(0, color="black", linewidth=1)
plt.axvline(0, color="black", linewidth=1)

# kuadran ideal: x<0 (kemiskinan turun), y>0 (IPM naik)
plt.title("Quadrant Plot: Δ Kemiskinan vs Δ IPM (S3)")
plt.xlabel("Δ Kemiskinan S3 (lebih negatif lebih baik)")
plt.ylabel("Δ IPM S3 (lebih positif lebih baik)")
plt.grid(alpha=0.2)
plt.tight_layout()
plt.show()

# ----------------------------------------
# 6) Baseline vs S1/S2/S3 per indikator
# ----------------------------------------
scenario_labels = ["s1", "s2", "s3"]
pairs = [
    ("poor_rate_baseline", "poor_rate", "Kemiskinan"),
    ("IPM_baseline", "IPM", "IPM"),
    ("TPT_baseline", "TPT", "TPT"),
]

for base_col, pref, title in pairs:
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    for ax, s in zip(axes, scenario_labels):
        ycol = f"{pref}_{s}"
        ax.scatter(output[base_col], output[ycol], alpha=0.75)
        mn = min(output[base_col].min(), output[ycol].min())
        mx = max(output[base_col].max(), output[ycol].max())
        ax.plot([mn, mx], [mn, mx], "r--", linewidth=1)
        ax.set_title(f"Baseline vs {s.upper()} ({title})")
        ax.set_xlabel("Baseline")
        ax.set_ylabel(s.upper())
        ax.grid(alpha=0.2)
    plt.tight_layout()
    plt.show()

# ----------------------------------------
# 7) Waterfall sederhana kontribusi skenario
# ----------------------------------------
# Pendekatan: gunakan rata-rata delta nasional sebagai langkah perubal
avg_delta = {
    "S1": output["delta_poor_rate_s1"].mean(),
    "S2": output["delta_poor_rate_s2"].mean(),
    "S3": output["delta_poor_rate_s3"].mean(),
}
```

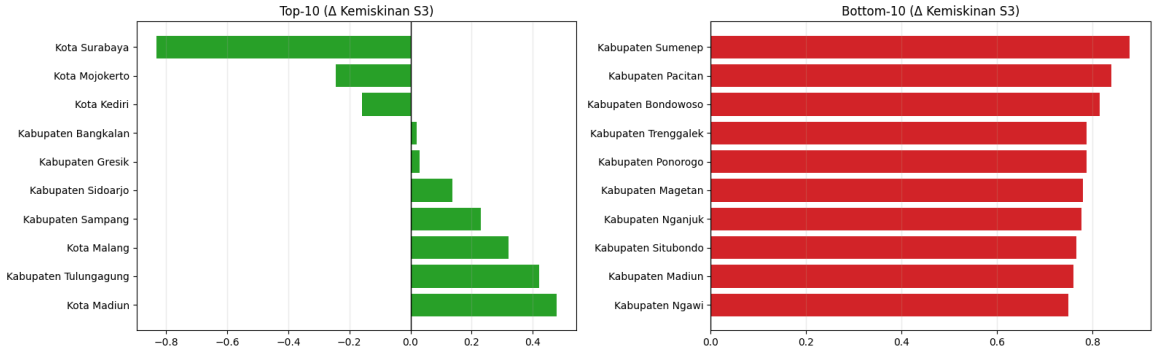
```
waterfall_steps = pd.Series(avg_delta)
cum = waterfall_steps.cumsum()
start_vals = np.r_[0, cum.values[:-1]]

plt.figure(figsize=(8, 5))
colors = ["#1f77b4" if v >= 0 else "#2ca02c" for v in waterfall_steps
for i, (label, val) in enumerate(waterfall_steps.items()):
    plt.bar(label, val, bottom=start_vals[i], color=colors[i])
    plt.text(i, start_vals[i] + val / 2, f"{val:.3f}", ha="center", v

plt.axhline(0, color="black", linewidth=1)
plt.title("Waterfall Rata-rata Δ Kemiskinan per Skenario")
plt.ylabel("Delta kumulatif")
plt.tight_layout()
plt.show()

# ----------------------------------------
# 8) Heatmap korelasi fitur lag dan target
# ----------------------------------------
if "panel_lagged" in globals() and not panel_lagged.empty:
    corr_cols = [
        "poor_rate_1000",
        "IPM",
        "TPT",
        "poor_rate_1000_lag1",
        "IPM_lag1",
        "TPT_lag1",
        "PAD_per_capita_lag1",
        "belanja_per_capita_lag1",
        "PDRB_per_kapita_ADHK_lag1",
    ]
    corr = panel_lagged[corr_cols].corr()

    plt.figure(figsize=(10, 8))
    im = plt.imshow(corr.values, cmap="coolwarm", vmin=-1, vmax=1)
    plt.xticks(range(len(corr_cols)), corr_cols, rotation=60, ha="rig
    plt.yticks(range(len(corr_cols)), corr_cols)
    plt.colorbar(im, fraction=0.046, pad=0.04)

    for i in range(len(corr_cols)):
        for j in range(len(corr_cols)):
            plt.text(j, i, f"{corr.values[i, j]:.2f}", ha="center", v

    plt.title("Heatmap Korelasi Variabel Lag dan Target")
    plt.tight_layout()
    plt.show()
else:
    print("Lewati heatmap korelasi: variabel 'panel_lagged' belum ter

# ----------------------------------------
# 9) Koefisien Ridge per model
# ----------------------------------------
if "coef_tables" in globals() and isinstance(coef_tables, dict) and l
    n_model = len(coef_tables)
    fig, axes = plt.subplots(1, n_model, figsize=(6 * n_model, 5))
```

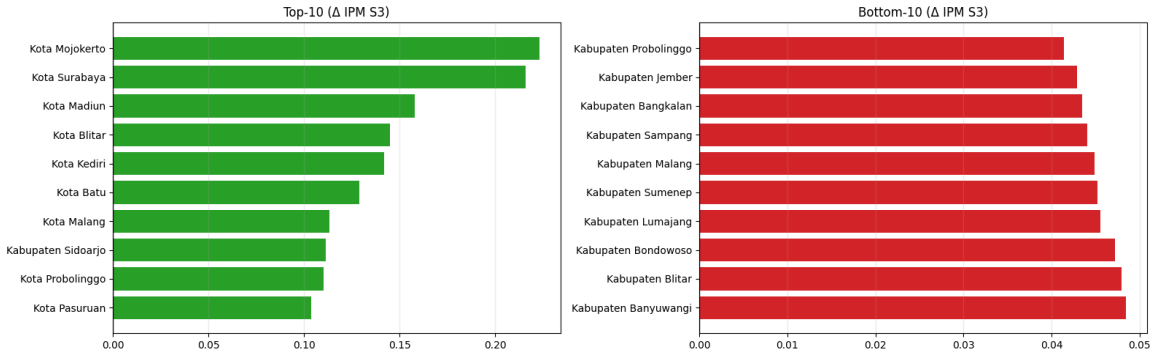
```
    if n_model == 1:
        axes = [axes]

    for ax, (model_name, coef_df) in zip(axes, coef_tables.items()):
        temp = coef_df.sort_values("coefficient")
        ax.barh(temp["feature"], temp["coefficient"])
        ax.axvline(0, color="black", linewidth=1)
        ax.set_title(f"Koefisien Ridge: {model_name}")
        ax.grid(axis="x", alpha=0.2)

    plt.tight_layout()
    plt.show()
else:
    print("Lewati koefisien: 'coef_tables' belum tersedia. Jalankan c

# ----------------------------------------
# 10) Kinerja model (RMSE/MAE)
# ----------------------------------------
if "metrics_df" in globals() and not metrics_df.empty:
    metric_plot = metrics_df.copy()

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    axes[0].bar(metric_plot["model"], metric_plot["cv_rmse_mean"], ye
    axes[0].set_title("CV RMSE (mean ± std)")
    axes[0].set_ylabel("RMSE")
    axes[0].grid(axis="y", alpha=0.2)

    axes[1].bar(metric_plot["model"], metric_plot["cv_mae_mean"], yer
    axes[1].set_title("CV MAE (mean ± std)")
    axes[1].set_ylabel("MAE")
    axes[1].grid(axis="y", alpha=0.2)

    plt.tight_layout()
    plt.show()
else:
    print("Lewati kinerja model: variabel 'metrics_df' belum tersedia

print("Selesai: paket visualisasi lengkap berhasil dijalankan.")
```

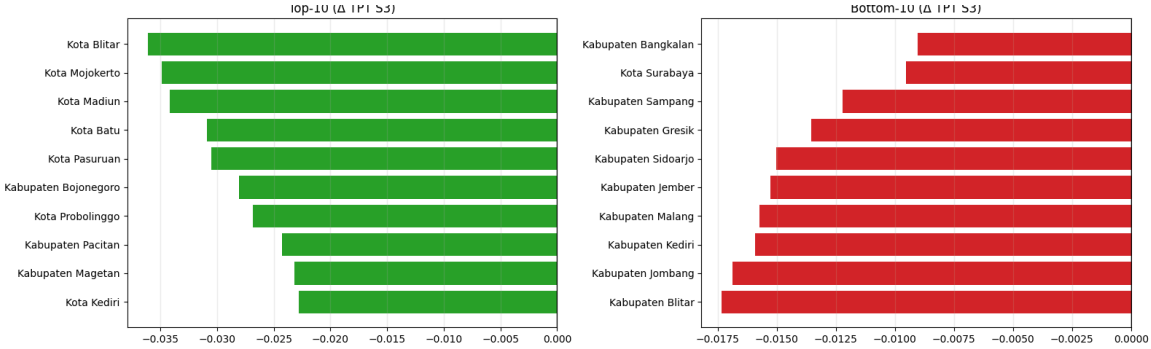

Tren Time Series Rata-rata Kabupaten/Kota
Tren Rata-rata poor_rate_1000
Tren Rata-rata IPM
Tren Rata-rata TPT
Tren Rata-rata PAD_per_capita
Tren Rata-rata belanja_per_capita
Tren Rata-rata PDRB_per_kapita_ADHK
Tahun

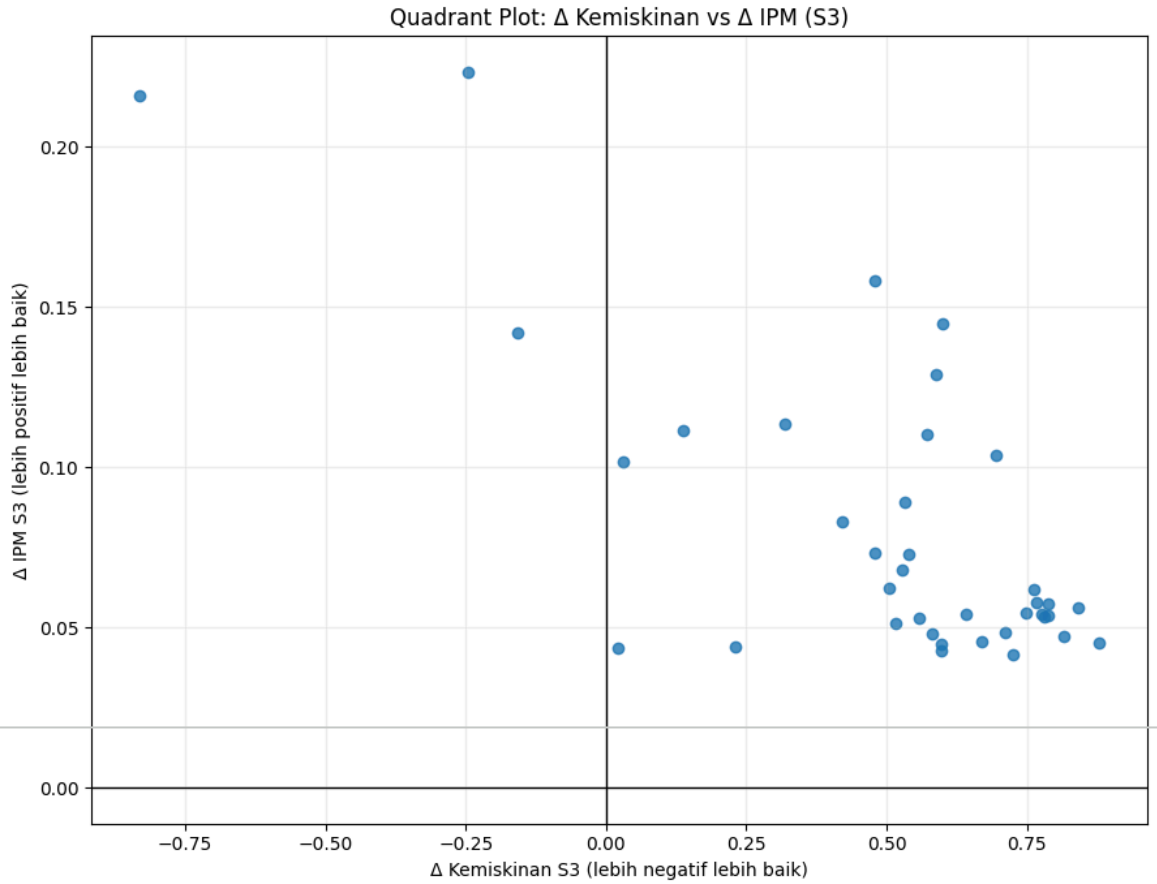

Pergeseran Distribusi Antar Tahun
Distribusi poor_rate_1000 per Tahun
Distribusi IPM per Tahun
Distribusi TPT per Tahun
Tahun

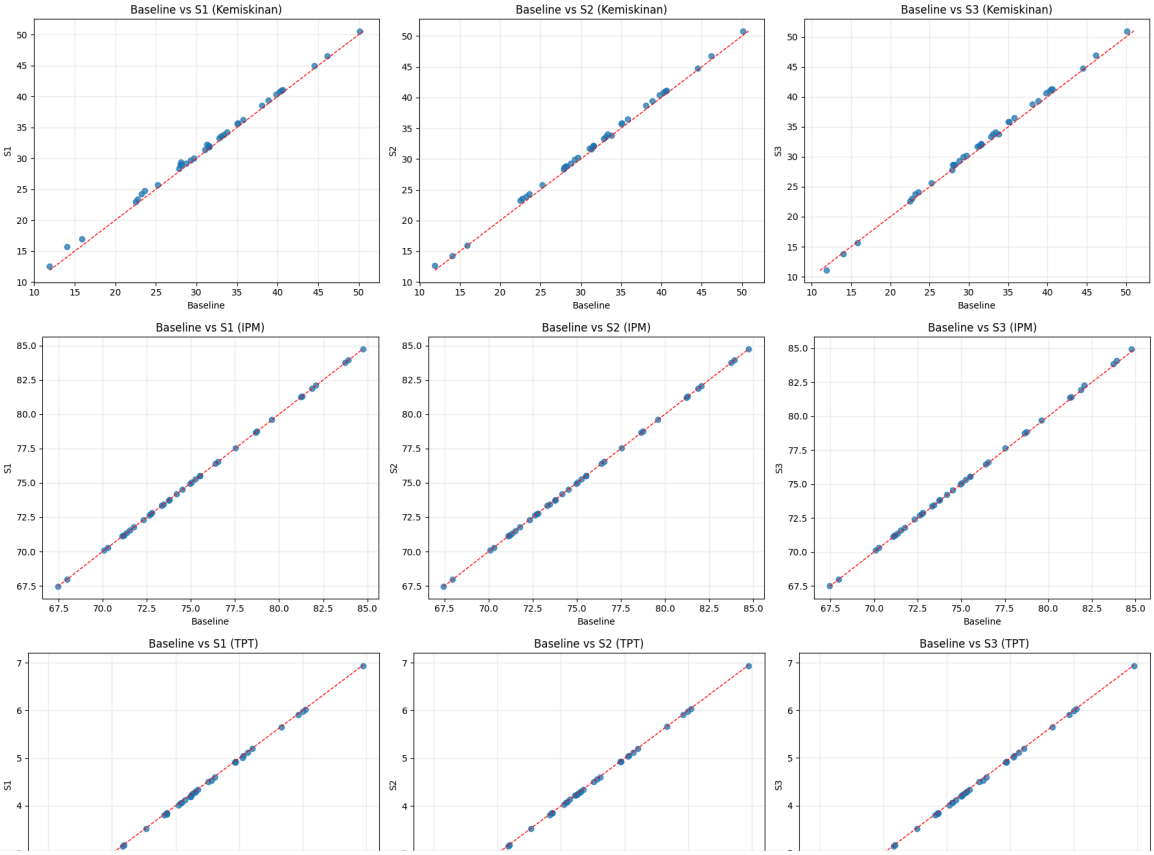

Choropleth Multipanel Dampak S3
Δ Kemiskinan S3
Δ IPM S3
Δ TPT S3


Top-10 (Δ Kemiskinan S3)
Kota Surabaya
Kota Mojokerto
Kota Kediri
Kabupaten Bangkalan
Kabupaten Gresik
Kabupaten Sidoarjo
Kabupaten Sampang
Kota Malang
Kabupaten Tulungagung
Kota Madiun
Bottom-10 (Δ Kemiskinan S3)
Kabupaten Sumenep
Kabupaten Pacitan
Kabupaten Bondowoso
Kabupaten Trenggalek
Kabupaten Ponorogo
Kabupaten Magetan
Kabupaten Nganjuk
Kabupaten Situbondo
Kabupaten Madiun
Kabupaten Ngawi


Top-10 (Δ IPM S3)
Kota Mojokerto
Kota Surabaya
Kota Madiun
Kota Blitar
Kota Kediri
Kota Batu
Kota Malang
Kabupaten Sidoarjo
Kota Probolinggo
Kota Pasuruan
Bottom-10 (Δ IPM S3)
Kabupaten Probolinggo
Kabupaten Jember
Kabupaten Bangkalan
Kabupaten Sampang
Kabupaten Malang
Kabupaten Sumenep
Kabupaten Lumajang
Kabupaten Bondowoso
Kabupaten Blitar
Kabupaten Banyuwangi

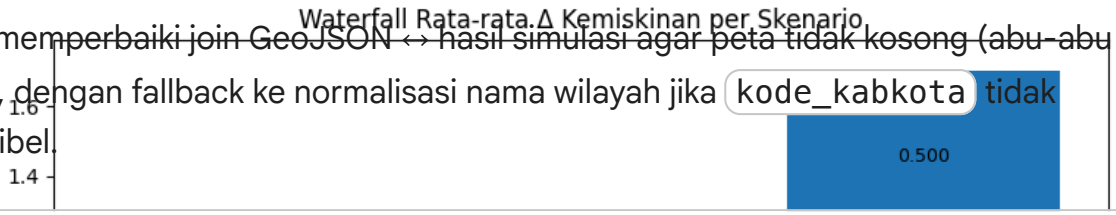

Top-10 (Δ TPT S3)
Kota Blitar
Kota Mojokerto
Kota Madiun
Kota Batu
Kota Pasuruan
Kabupaten Bojonegoro
Kota Probolinggo
Kabupaten Pacitan
Kabupaten Magetan
Kota Kediri
−0.035 −0.030 −0.025 −0.020 −0.015 −0.010 −0.005 0.000
Bottom-10 (Δ TPT S3)
Kabupaten Bangkalan
Kota Surabaya
Kabupaten Sampang
Kabupaten Gresik
Kabupaten Sidoarjo
Kabupaten Jember
Kabupaten Malang
Kabupaten Kediri
Kabupaten Jombang
Kabupaten Blitar
−0.0175 −0.0150 −0.0125 −0.0100 −0.0075 −0.0050 −0.0025 0.0000


Quadrant Plot: Δ Kemiskinan vs Δ IPM (S3)
Δ IPM S3 (lebih positif lebih baik)
0.20
0.15
0.10
0.05
0.00
−0.75 −0.50 −0.25 0.00 0.25 0.50 0.75
Δ Kemiskinan S3 (lebih negatif lebih baik)


Baseline vs S1 (Kemiskinan)
Baseline vs S2 (Kemiskinan)
Baseline vs S3 (Kemiskinan)
Baseline vs S1 (IPM)
Baseline vs S2 (IPM)
Baseline vs S3 (IPM)
Baseline vs S1 (TPT)
Baseline vs S2 (TPT)
Baseline vs S3 (TPT)
Baseline
S1
S2
S3

Perbaikan Choropleth (Robust Join Key)

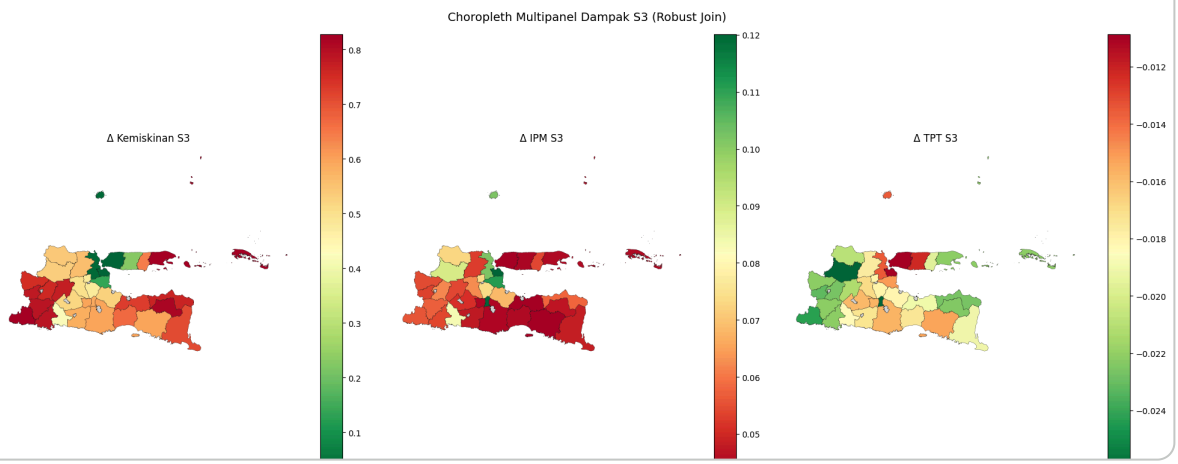
Cell ini memperbaiki join GeoJSON ↔ hasil simulasi agar peta tidak kosong (abu-abu semua), dengan fallback ke normalisasi nama wilayah jika `kode_kabkota` tidak kompatibel.




```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

if "ROOT" not in globals():
    ROOT = Path(".").resolve()
if "output" not in globals():
    output = pd.read_csv(ROOT / "policy_simulation_output.csv")

path_geojson = ROOT / "Kabupaten-Kota (Provinsi Jawa Timur).geojson"

if not path_geojson.exists():
    print(f"GeoJSON tidak ditemukan: {path_geojson}")
else:
    try:
        import geopandas as gpd

        gdf = gpd.read_file(path_geojson)
        out = output.copy()

        def _norm_name(x):
            s = "" if pd.isna(x) else str(x).strip().lower()
            s = re.sub(r"\s+", " ", s)
            s = re.sub(r"^kab\.?\s+", "kabupaten ", s)
            s = re.sub(r"^kota\.?\s+", "kota ", s)
            return s

        gdf_plot = None
        join_note = ""

        if "kode_kabkota" in gdf.columns:
            left = gdf["kode_kabkota"].astype(str).str.strip()
            right = out["kode_kabkota"].astype(str).str.strip()
            overlap = len(set(left).intersection(set(right)))
            if overlap >= 10:
                gdf_plot = gdf.merge(
                    out[["kode_kabkota", "delta_poor_rate_s3", "delta_
                    on="kode_kabkota",
                    how="left",
                )
                join_note = f"Join key: kode_kabkota (overlap={overla

        if gdf_plot is None:
            gdf_tmp = gdf.copy()
```

```
        # Prioritas: struktur GADM (NAME_2 + TYPE_2)
        if {"NAME_2", "TYPE_2"}.issubset(gdf_tmp.columns):
            gdf_tmp["join_name"] = (
                gdf_tmp["TYPE_2"].astype(str).str.strip() + " " +
            ).apply(_norm_name)
            join_note = "Join key fallback: TYPE_2 + NAME_2"
        else:
            name_candidates = [
                c for c in gdf_tmp.columns if c != "geometry" and
            ]
            if not name_candidates:
                raise KeyError("Tidak ada kandidat kolom nama wil

            def _score_col(col):
                lc = col.lower()
                score = 0
                if lc in {"name_2", "name2", "nama_kabkota", "wad
                    score += 100
                if any(t in lc for t in ["name", "nama", "kab", "
                    score += 30
                score += int(gdf_tmp[col].nunique(dropna=True))
                return score

            pick = sorted(name_candidates, key=_score_col, revers
            gdf_tmp["join_name"] = gdf_tmp[pick].apply(_norm_name
            join_note = f"Join key fallback: nama wilayah ({pick}

        if "harmonize_region_name" in globals() and "canonical_lo
            gdf_tmp["join_name"] = gdf_tmp["join_name"].apply(
                lambda x: harmonize_region_name(x, canonical_look
            )
            gdf_tmp["join_name"] = gdf_tmp["join_name"].apply(lam

        out["join_name"] = out["nama_kabkota"].apply(_norm_name)

        left_df = gdf_tmp.copy()
        right_df = out[["join_name", "delta_poor_rate_s3", "delta_

        left_df["join_name"] = left_df["join_name"].astype("strin
        right_df["join_name"] = right_df["join_name"].astype("str

        gdf_plot = left_df.merge(right_df, on="join_name", how="l

    map_specs = [
        ("delta_poor_rate_s3", "Δ Kemiskinan S3", "RdYlGn_r"),
        ("delta_IPM_s3", "Δ IPM S3", "RdYlGn"),
        ("delta_TPT_s3", "Δ TPT S3", "RdYlGn_r"),
    ]

    fig, axes = plt.subplots(1, 3, figsize=(20, 8))
    for ax, (col, title, cmap) in zip(axes, map_specs):
        valid = gdf_plot[col].dropna()
        vmin = float(valid.quantile(0.05)) if len(valid) else Non
        vmax = float(valid.quantile(0.95)) if len(valid) else Non
```

```
            gdf_plot.plot(
                column=col,
                cmap=cmap,
                linewidth=0.2,
                edgecolor="black",
                legend=True,
                vmin=vmin,
                vmax=vmax,
                missing_kwds={"color": "lightgrey", "label": "Data NA
                ax=ax,
            )
            ax.set_title(title)
            ax.set_axis_off()

        plt.suptitle("Choropleth Multipanel Dampak S3 (Robust Join)",
        plt.tight_layout()
        plt.show()

        matched = gdf_plot["delta_poor_rate_s3"].notna().sum()
        total = len(gdf_plot)
        print(join_note)
        print(f"Wilayah match (delta_poor_rate_s3): {matched}/{total}

    except ImportError:
        print("geopandas belum terpasang di kernel aktif.")
```



Visualisasi Indeks Pembangunan Manusia (IPM) Jawa Timur 2020–2024

```
import re
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from matplotlib.colors import Normalize
```

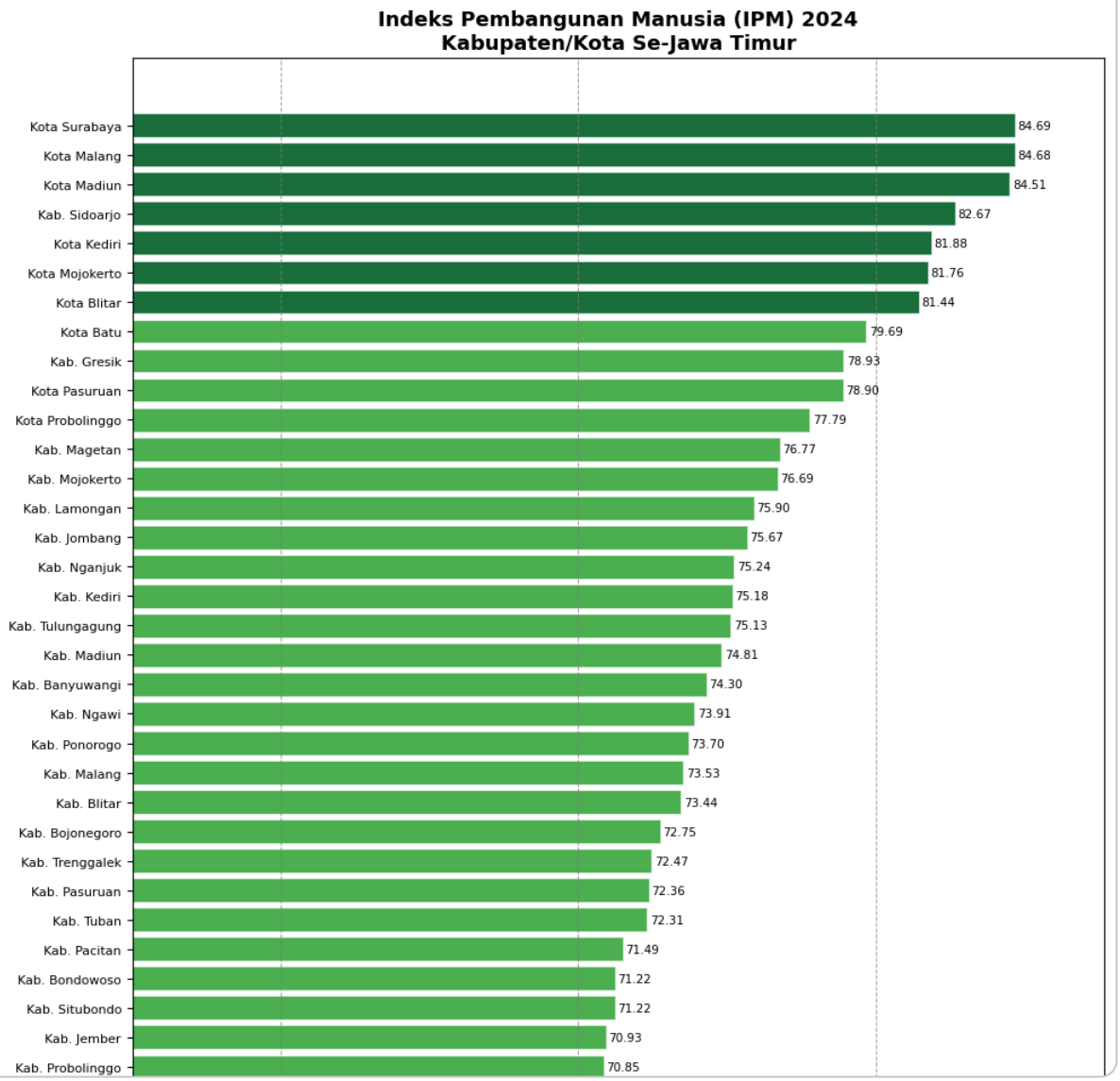
```
from matplotlib.cm import ScalarMappable

warnings.filterwarnings("ignore")

# ── 1. Load & Parse IPM files ──────────────────────────────────────────
IPM_DIR = Path(".").resolve() / "IPM"

def load_ipm(ipm_dir: Path) -> pd.DataFrame:
    records = []
    for yr in range(2020, 2025):
        path = ipm_dir / f"ipm {yr}.xlsx"
        if not path.exists():
            print(f"File tidak ditemukan: {path}")
            continue
        df = pd.read_excel(path, header=None)
        for _, row in df.iterrows():
            region = str(row.iloc[0]).strip() if pd.notna(row.iloc[0]
            value  = row.iloc[1]
            if not region or region.lower().startswith("kabupaten/kot
                continue
            try:
                ipm_val = float(str(value).replace(",", "."))
            except ValueError:
                continue
            records.append({"region": region, "tahun": yr, "IPM": ipm
    return pd.DataFrame(records)

df_ipm = load_ipm(IPM_DIR)

# Clean: remove total/summary rows (pure numeric names or very short)
df_ipm = df_ipm[df_ipm["region"].str.match(r"(?i)^(kabupaten|kota)\s+
df_ipm["region_short"] = (
    df_ipm["region"]
    .str.replace(r"^Kabupaten\s+", "Kab. ", regex=True)
    .str.replace(r"^Kota\s+",      "Kota ", regex=True)
)

print(f"Jumlah wilayah: {df_ipm['region'].nunique()}")
print(f"Tahun: {sorted(df_ipm['tahun'].unique())}")
display(df_ipm.head())
```

```
Jumlah wilayah: 38
Tahun: [np.int64(2020), np.int64(2021), np.int64(2022), np.int64(2023)
```

	region	tahun	IPM	region_short
0	Kabupaten Pacitan	2020	69.28	Kab. Pacitan
1	Kabupaten Ponorogo	2020	71.57	Kab. Ponorogo
2	Kabupaten Trenggalek	2020	70.10	Kab. Trenggalek
3	Kabupaten Tulungagung	2020	73.15	Kab. Tulungagung
4	Kabupaten Blitar	2020	71.02	Kab. Blitar

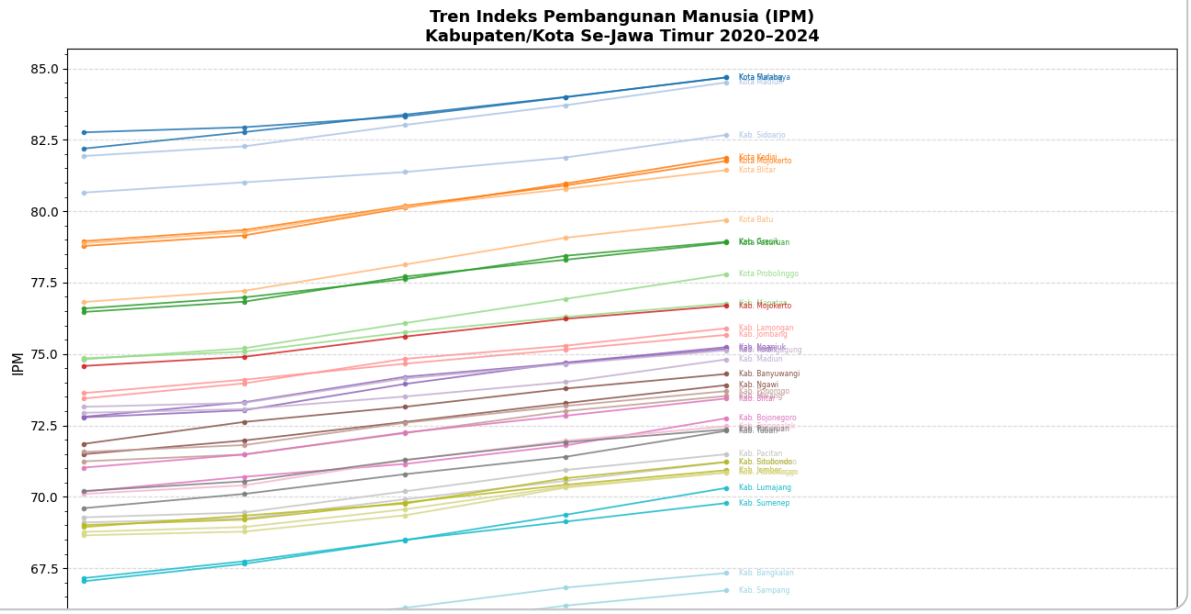
```
# ── 2. Bar Chart: IPM 2024 (sorted, colour-coded by category) ──────
df_2024 = df_ipm[df_ipm["tahun"] == 2024].sort_values("IPM", ascendin

def ipm_color(val):
    if val >= 80:   return "#1a6e3c"  # tinggi
    if val >= 70:   return "#4caf50"  # menengah atas
    if val >= 60:   return "#ff9800"  # menengah bawah
    return "#d32f2f"                  # rendah

colors = [ipm_color(v) for v in df_2024["IPM"]]

fig, ax = plt.subplots(figsize=(10, 12))
bars = ax.barh(df_2024["region_short"], df_2024["IPM"], color=colors,

# Value labels
for bar, val in zip(bars, df_2024["IPM"]):
    ax.text(bar.get_width() + 0.1, bar.get_y() + bar.get_height() / 2
            f"{val:.2f}", va="center", ha="left", fontsize=7.5)

# Reference lines
for ref, lbl in [(60, "60"), (70, "70 (Menengah Atas)"), (80, "80 (Ti
    ax.axvline(ref, color="grey", linewidth=0.7, linestyle="--", alph
    ax.text(ref + 0.1, ax.get_ylim()[0], lbl, fontsize=7, color="grey

ax.set_xlabel("IPM", fontsize=11)
ax.set_xlim(55, max(df_2024["IPM"]) + 3)
ax.set_title("Indeks Pembangunan Manusia (IPM) 2024\nKabupaten/Kota S
             fontsize=13, fontweight="bold")
ax.tick_params(axis="y", labelsize=8)

# Legend
from matplotlib.patches import Patch
legend_els = [
    Patch(facecolor="#1a6e3c", label="≥ 80 (Tinggi)"),
    Patch(facecolor="#4caf50", label="70–79 (Menengah Atas)"),
    Patch(facecolor="#ff9800", label="60–69 (Menengah Bawah)"),
    Patch(facecolor="#d32f2f", label="< 60 (Rendah)"),
]
ax.legend(handles=legend_els, fontsize=8, loc="lower right")
plt.tight_layout()
```

```
plt.savefig("ipm_bar_2024.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: ipm_bar_2024.png")
```



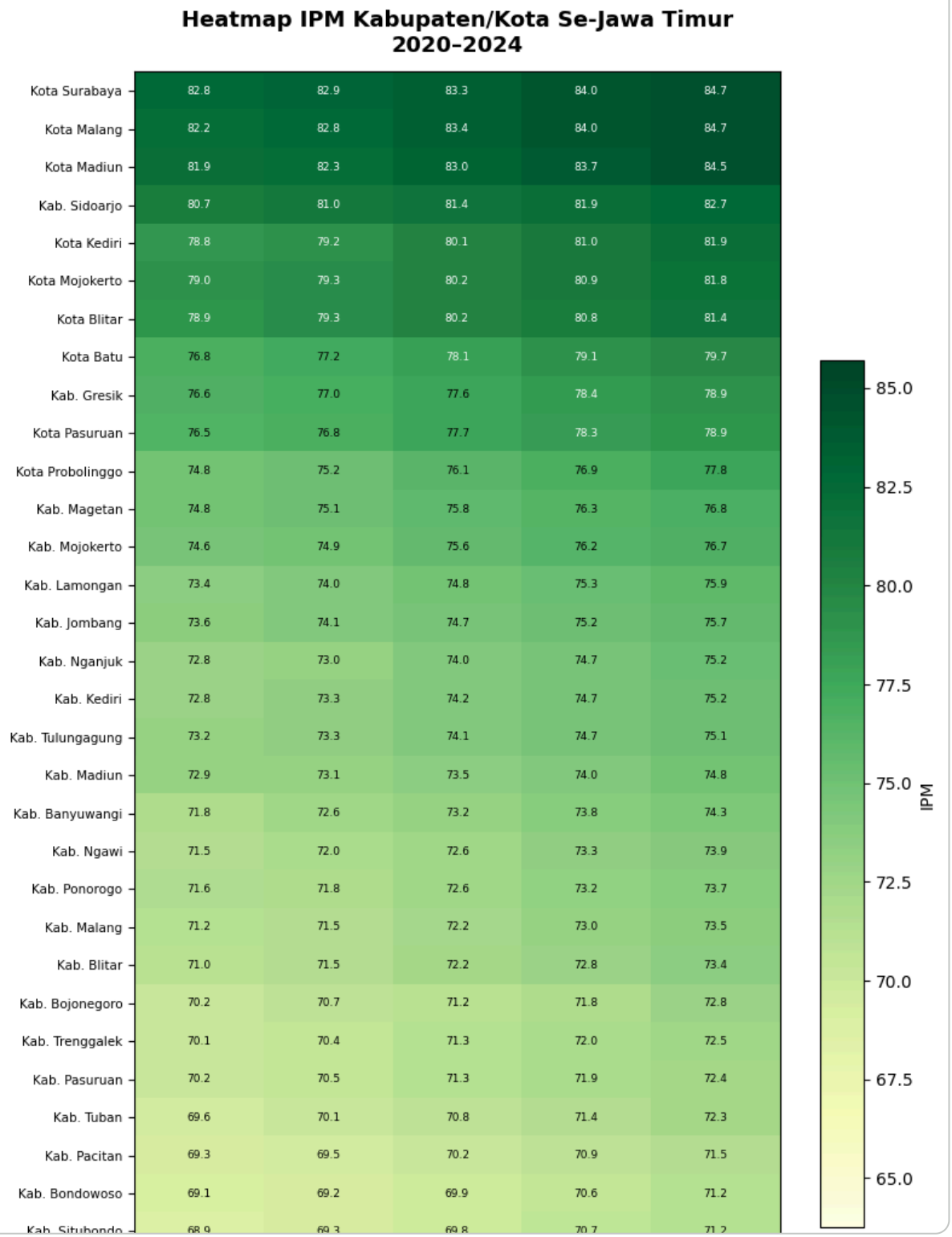

```
# ── 3. Line Chart: Tren IPM 2020–2024 per Wilayah ──────────────
pivot = df_ipm.pivot(index="region_short", columns="tahun", values="I

fig, ax = plt.subplots(figsize=(13, 8))

cmap = plt.cm.get_cmap("tab20", len(pivot))
years = sorted(df_ipm["tahun"].unique())

for i, (region, row) in enumerate(pivot.iterrows()):
    vals = row[years].values
    ax.plot(years, vals, marker="o", markersize=3, linewidth=1.3,
            color=cmap(i), alpha=0.85, label=region)
    # Label right end
    ax.text(years[-1] + 0.08, vals[-1], region, fontsize=5.5,
            va="center", color=cmap(i))

ax.set_xticks(years)
ax.set_xlabel("Tahun", fontsize=11)
ax.set_ylabel("IPM", fontsize=11)
```

```
ax.set_title("Tren Indeks Pembangunan Manusia (IPM)\nKabupaten/Kota S
             fontsize=13, fontweight="bold")
ax.yaxis.set_minor_locator(mticker.AutoMinorLocator(5))
ax.grid(axis="y", which="major", linestyle="--", alpha=0.4)
ax.set_xlim(years[0] - 0.1, years[-1] + 2.8)
plt.tight_layout()
plt.savefig("ipm_trend_lines.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: ipm_trend_lines.png")
```




```
# ── 4. Heatmap: IPM semua wilayah × tahun ──────────────────────────
import matplotlib.pyplot as plt
import numpy as np

pivot_heat = df_ipm.pivot(index="region_short", columns="tahun", valu

fig, ax = plt.subplots(figsize=(8, 13))
im = ax.imshow(pivot_heat.values, aspect="auto", cmap="YlGn",
               vmin=pivot_heat.values.min() - 1, vmax=pivot_heat.valu

# Axes labels
ax.set_xticks(range(len(pivot_heat.columns)))
ax.set_xticklabels(pivot_heat.columns, fontsize=10)
ax.set_yticks(range(len(pivot_heat.index)))
ax.set_yticklabels(pivot_heat.index, fontsize=7.5)

# Cell annotations
for i in range(len(pivot_heat.index)):
    for j in range(len(pivot_heat.columns)):
        val = pivot_heat.values[i, j]
        if not np.isnan(val):
            txt_color = "black" if val < 78 else "white"
            ax.text(j, i, f"{val:.1f}", ha="center", va="center",
                    fontsize=6.5, color=txt_color)

plt.colorbar(im, ax=ax, label="IPM", shrink=0.6)
ax.set_title("Heatmap IPM Kabupaten/Kota Se-Jawa Timur\n2020–2024",
```

```
                fontsize=13, fontweight="bold", pad=12)
plt.tight_layout()
plt.savefig("ipm_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: ipm_heatmap.png")
```

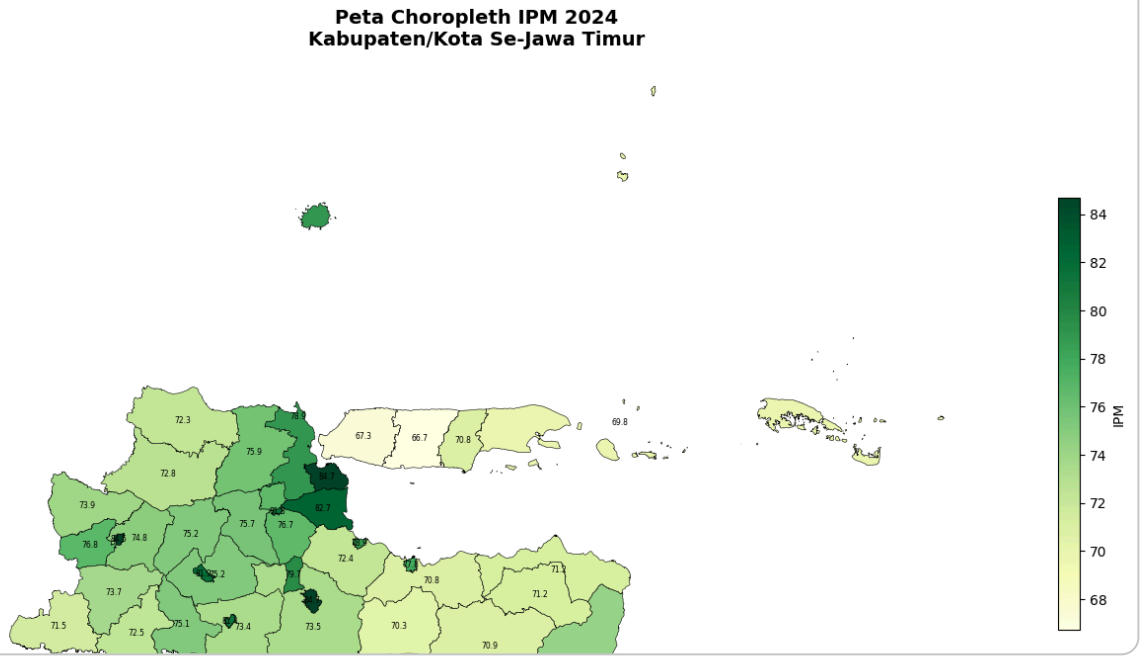



```
# ── 5. Choropleth Peta IPM 2024 ──────────────────────────────
try:
    import geopandas as gpd
    import re as _re

    gdf = gpd.read_file("Kabupaten-Kota (Provinsi Jawa Timur).geojson
```

```
def norm(s):
    s = str(s).strip().lower()
    s = _re.sub(r"\s+", " ", s)
    return s

# Build canonical name from TYPE_2 + NAME_2
# Some Kota rows have "Kota X" in NAME_2 → strip leading "kota " 
def build_gdf_key(row):
    t = str(row["TYPE_2"]).strip()   # "Kabupaten" or "Kota"
    n = str(row["NAME_2"]).strip()
    # Remove leading type prefix from name if already present
    n_clean = _re.sub(r"^(?:kabupaten|kota)\s+", "", n, flags=_re
    return norm(f"{t} {n_clean}")

gdf["join_key"] = gdf.apply(build_gdf_key, axis=1)

df_2024_choropleth = df_ipm[df_ipm["tahun"] == 2024].copy()
df_2024_choropleth["join_key"] = df_2024_choropleth["region"].app

gdf_merged = gdf.merge(df_2024_choropleth[["join_key", "IPM"]], o

matched = gdf_merged["IPM"].notna().sum()
total   = len(gdf_merged)
print(f"Match: {matched}/{total}")
if matched < total:
    unmatched_geo = gdf_merged[gdf_merged["IPM"].isna()]["join_ke
    print("Unmatched GeoJSON keys:", unmatched_geo)

fig, ax = plt.subplots(1, 1, figsize=(14, 8))
gdf_merged.plot(
    column="IPM",
    cmap="YlGn",
    linewidth=0.4,
    edgecolor="black",
    legend=True,
    legend_kwds={"label": "IPM", "orientation": "vertical", "shri
    missing_kwds={"color": "lightgrey", "label": "Data NA"},
    ax=ax,
)

# Centroid labels
for _, row in gdf_merged.iterrows():
    if pd.notna(row.get("IPM")):
        cx, cy = row.geometry.centroid.x, row.geometry.centroid.y
        ax.text(cx, cy, f"{row['IPM']:.1f}", fontsize=5.5,
                ha="center", va="center", color="black")

ax.set_title("Peta Choropleth IPM 2024\nKabupaten/Kota Se-Jawa Ti
             fontsize=14, fontweight="bold")
ax.set_axis_off()
plt.tight_layout()
plt.savefig("ipm_choropleth_2024.png", dpi=150, bbox_inches="tigh
plt.show()
print("Saved: ipm_choropleth_2024.png")
```

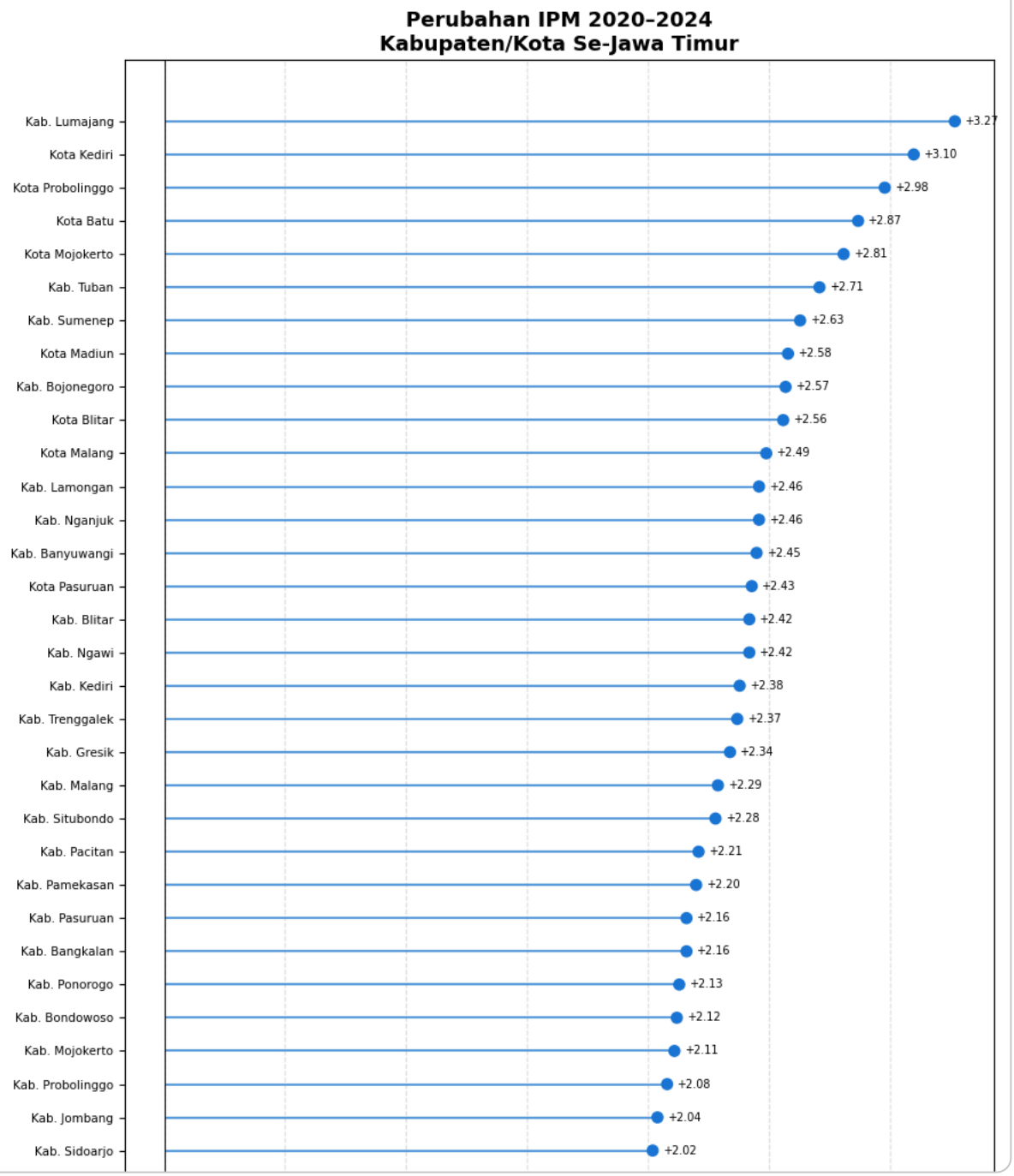
```
except ImportError:
    print("geopandas tidak terpasang. Lewati peta choropleth.")
    print("Install dengan: pip install geopandas")
```

```
Match: 38/38
```




```
# ── 6. Perubahan IPM 2020→2024 (Lollipop Chart) ──────────────────────
df_delta = df_ipm.pivot(index="region_short", columns="tahun", values=
df_delta = df_delta[[2020, 2024]].dropna()
df_delta["delta"] = df_delta[2024] - df_delta[2020]
df_delta = df_delta.sort_values("delta", ascending=True)

colors_delta = ["#d32f2f" if d < 0 else "#1976d2" for d in df_delta["

fig, ax = plt.subplots(figsize=(9, 13))
y_pos = range(len(df_delta))

ax.hlines(y_pos, 0, df_delta["delta"].values, colors=colors_delta, li
ax.scatter(df_delta["delta"].values, y_pos, color=colors_delta, s=45,

ax.axvline(0, color="black", linewidth=0.8)
ax.set_yticks(list(y_pos))
ax.set_yticklabels(df_delta.index, fontsize=7.5)
ax.set_xlabel("Perubahan IPM (poin)", fontsize=11)
ax.set_title("Perubahan IPM 2020–2024\nKabupaten/Kota Se-Jawa Timur",
             fontsize=13, fontweight="bold")
ax.xaxis.set_minor_locator(mticker.AutoMinorLocator(5))
ax.grid(axis="x", which="major", linestyle="--", alpha=0.35)

# Label values
for val, yp in zip(df_delta["delta"].values, y_pos):
    offset = 0.04 if val >= 0 else -0.04
    ha = "left" if val >= 0 else "right"
    ax.text(val + offset, yp, f"{val:+.2f}", va="center", ha=ha, font

plt.tight_layout()
```

```
plt.savefig("ipm_delta_2020_2024.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: ipm_delta_2020_2024.png")
```



Visualisasi Tingkat Pengangguran Terbuka (TPT) Jawa Timur 2020–2024

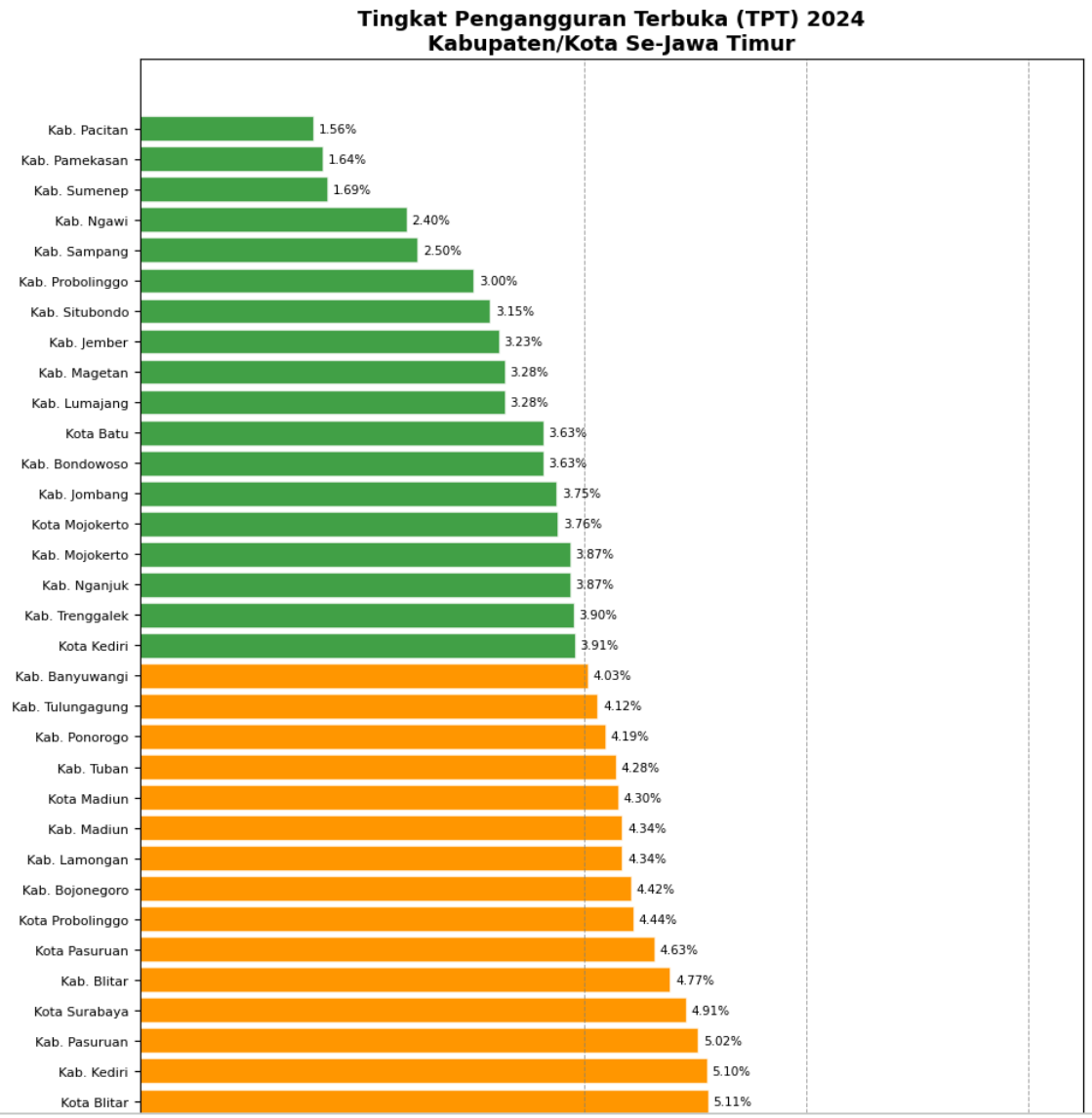
```
import re
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import matplotlib.ticker as mticker
from matplotlib.patches import Patch

warnings.filterwarnings("ignore")

# ── 1. Load & Parse TPT files ──────────────────────────────────────
TPT_DIR = Path(".").resolve() / "TPT"

def load_tpt(tpt_dir: Path) -> pd.DataFrame:
    records = []
    for yr in range(2020, 2025):
        path = tpt_dir / f"tpt {yr}.xlsx"
        if not path.exists():
            print(f"File tidak ditemukan: {path}")
            continue
        df = pd.read_excel(path, header=None)
        for _, row in df.iterrows():
            region = str(row.iloc[0]).strip() if pd.notna(row.iloc[0]
            value  = row.iloc[1]
            if not region or region.lower().startswith("kabupaten/kot
                continue
            try:
                tpt_val = float(str(value).replace(",", "."))
            except ValueError:
                continue
            records.append({"region": region, "tahun": yr, "TPT": tpt
    return pd.DataFrame(records)

df_tpt = load_tpt(TPT_DIR)
df_tpt = df_tpt[df_tpt["region"].str.match(r"(?i)^(kabupaten|kota)\s+
df_tpt["region_short"] = (
    df_tpt["region"]
    .str.replace(r"^Kabupaten\s+", "Kab. ", regex=True)
    .str.replace(r"^Kota\s+",      "Kota ", regex=True)
)

print(f"Jumlah wilayah: {df_tpt['region'].nunique()}")
print(f"Tahun: {sorted(df_tpt['tahun'].unique())}")
display(df_tpt.head())
```

```
Jumlah wilayah: 38
Tahun: [np.int64(2020), np.int64(2021), np.int64(2022), np.int64(2023)
```

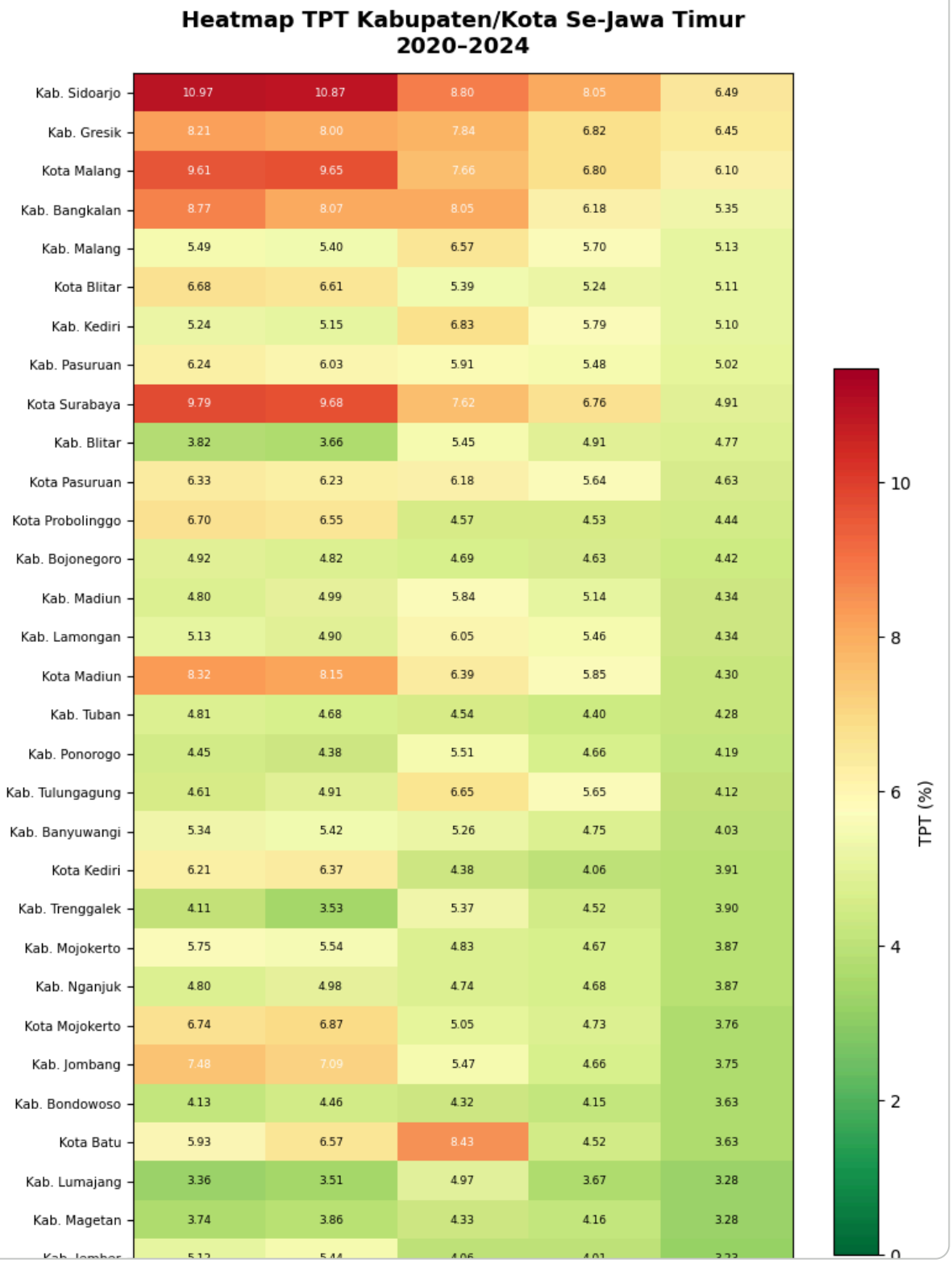
	region	tahun	TPT	region_short
0	Kabupaten Pacitan	2020	2.28	Kab. Pacitan
1	Kabupaten Ponorogo	2020	4.45	Kab. Ponorogo
2	Kabupaten Trenggalek	2020	4.11	Kab. Trenggalek
3	Kabupaten Tulungagung	2020	4.61	Kab. Tulungagung
4	Kabupaten Blitar	2020	3.82	Kab. Blitar

```
# ── 2. Bar Chart: TPT 2024 (sorted, colour-coded) ──────────────────
df_tpt_2024 = df_tpt[df_tpt["tahun"] == 2024].sort_values("TPT", ascen

def tpt_color(val):
    if val >= 8:   return "#b71c1c"   # sangat tinggi
    if val >= 6:   return "#ef5350"   # tinggi
    if val >= 4:   return "#ff9800"   # sedang
    return "#43a047"                  # rendah

colors_tpt = [tpt_color(v) for v in df_tpt_2024["TPT"]]

fig, ax = plt.subplots(figsize=(10, 12))
bars = ax.barh(df_tpt_2024["region_short"], df_tpt_2024["TPT"],
               color=colors_tpt, edgecolor="white", linewidth=0.4)

for bar, val in zip(bars, df_tpt_2024["TPT"]):
    ax.text(bar.get_width() + 0.05, bar.get_y() + bar.get_height() / 2
            f"{val:.2f}%", va="center", ha="left", fontsize=7.5)

for ref, lbl in [(4, "4%"), (6, "6% (Tinggi)"), (8, "8% (Sangat Tingg
    ax.axvline(ref, color="grey", linewidth=0.7, linestyle="--", alpha
    ax.text(ref + 0.05, ax.get_ylim()[0], lbl, fontsize=7, color="grey

ax.set_xlabel("TPT (%)", fontsize=11)
ax.set_xlim(0, df_tpt_2024["TPT"].max() + 2)
ax.set_title("Tingkat Pengangguran Terbuka (TPT) 2024\nKabupaten/Kota
             fontsize=13, fontweight="bold")
ax.tick_params(axis="y", labelsize=8)

legend_els = [
    Patch(facecolor="#b71c1c", label="≥ 8% (Sangat Tinggi)"),
    Patch(facecolor="#ef5350", label="6–8% (Tinggi)"),
    Patch(facecolor="#ff9800", label="4–6% (Sedang)"),
    Patch(facecolor="#43a047", label="< 4% (Rendah)"),
]
ax.legend(handles=legend_els, fontsize=8, loc="lower right")
plt.tight_layout()
plt.savefig("tpt_bar_2024.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: tpt_bar_2024.png")
```



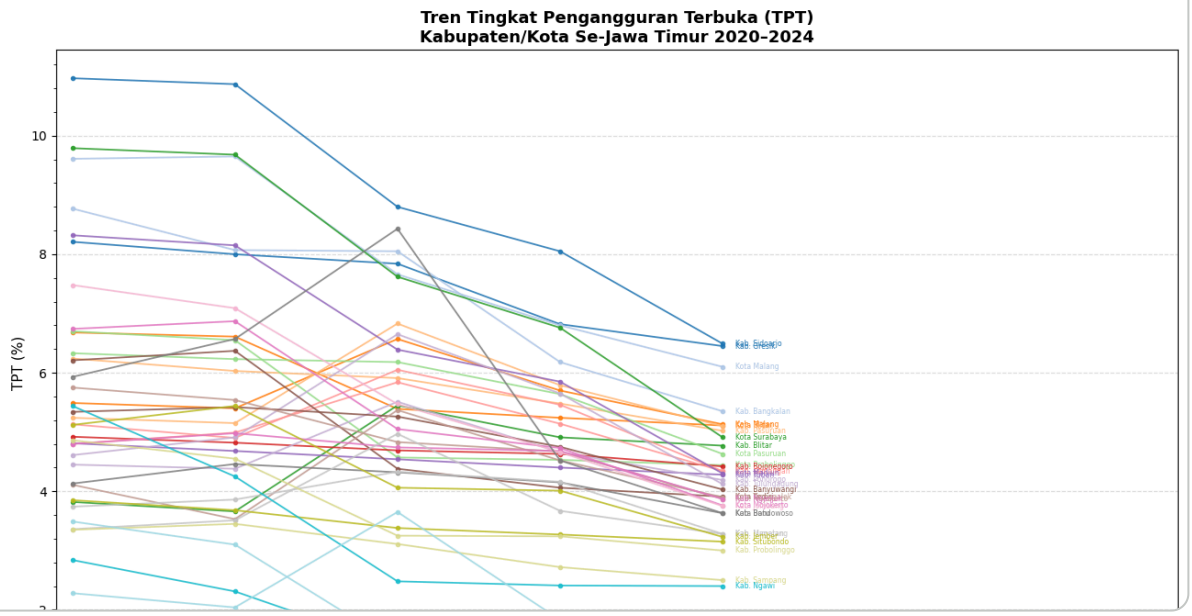

```
# ── 3. Heatmap: TPT semua wilayah × tahun ──────────────────────────
pivot_tpt = df_tpt.pivot(index="region_short", columns="tahun", value

fig, ax = plt.subplots(figsize=(8, 13))
im = ax.imshow(pivot_tpt.values, aspect="auto", cmap="RdYlGn_r",
               vmin=0, vmax=pivot_tpt.values.max() + 0.5)

ax.set_xticks(range(len(pivot_tpt.columns)))
ax.set_xticklabels(pivot_tpt.columns, fontsize=10)
ax.set_yticks(range(len(pivot_tpt.index)))
ax.set_yticklabels(pivot_tpt.index, fontsize=7.5)

for i in range(len(pivot_tpt.index)):
    for j in range(len(pivot_tpt.columns)):
        val = pivot_tpt.values[i, j]
        if not np.isnan(val):
            txt_color = "white" if val >= 7 else "black"
            ax.text(j, i, f"{val:.2f}", ha="center", va="center",
                    fontsize=6.5, color=txt_color)

plt.colorbar(im, ax=ax, label="TPT (%)", shrink=0.6)
ax.set_title("Heatmap TPT Kabupaten/Kota Se-Jawa Timur\n2020–2024",
             fontsize=13, fontweight="bold", pad=12)
```

```
plt.tight_layout()
plt.savefig("tpt_heatmap.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: tpt_heatmap.png")
```




```
# ── 4. Line Chart: Tren TPT 2020–2024 per Wilayah ──────────────
pivot_tpt_line = df_tpt.pivot(index="region_short", columns="tahun", 
years_tpt = sorted(df_tpt["tahun"].unique())

fig, ax = plt.subplots(figsize=(13, 8))
cmap_tpt = plt.cm.get_cmap("tab20", len(pivot_tpt_line))
```

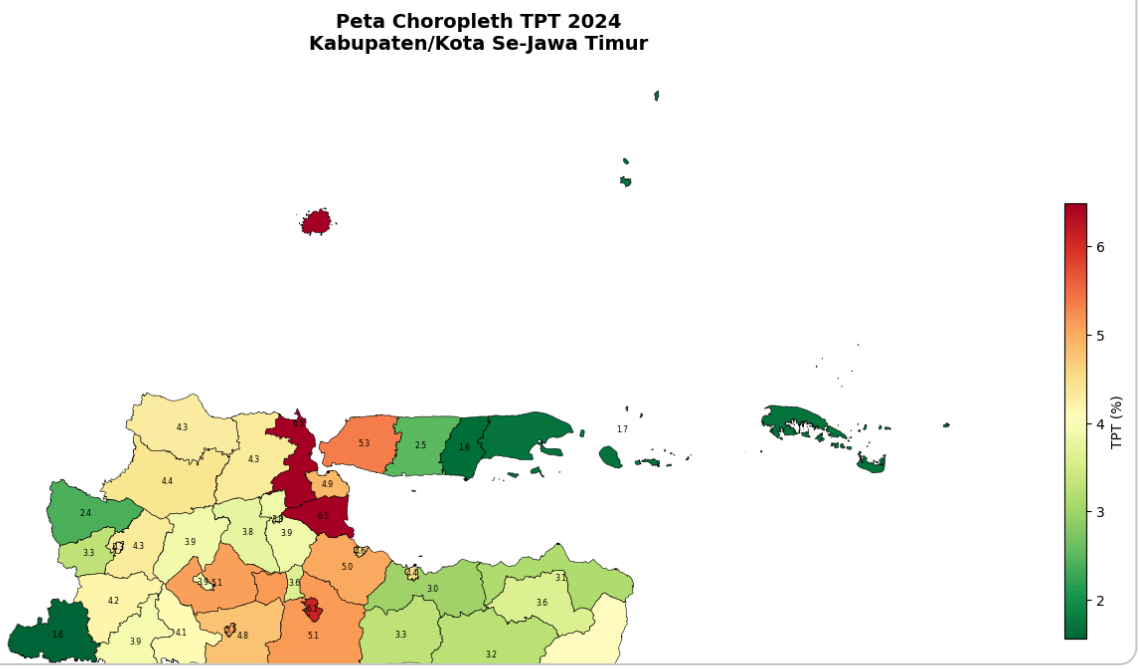
```
for i, (region, row) in enumerate(pivot_tpt_line.iterrows()):
    vals = row[years_tpt].values
    ax.plot(years_tpt, vals, marker="o", markersize=3, linewidth=1.3,
            color=cmap_tpt(i), alpha=0.85)
    ax.text(years_tpt[-1] + 0.08, vals[-1], region, fontsize=5.5,
            va="center", color=cmap_tpt(i))

ax.set_xticks(years_tpt)
ax.set_xlabel("Tahun", fontsize=11)
ax.set_ylabel("TPT (%)", fontsize=11)
ax.set_title("Tren Tingkat Pengangguran Terbuka (TPT)\nKabupaten/Kota
             fontsize=13, fontweight="bold")
ax.yaxis.set_minor_locator(mticker.AutoMinorLocator(5))
ax.grid(axis="y", which="major", linestyle="--", alpha=0.4)
ax.set_xlim(years_tpt[0] - 0.1, years_tpt[-1] + 2.8)
plt.tight_layout()
plt.savefig("tpt_trend_lines.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: tpt_trend_lines.png")
```




```
# ── 5. Choropleth Peta TPT 2024 ──────────────────────────────
try:
    import geopandas as gpd
    import re as _re

    gdf_tpt = gpd.read_file("Kabupaten-Kota (Provinsi Jawa Timur).geo

    def norm(s):
        s = str(s).strip().lower()
        return _re.sub(r"\s+", " ", s)

    def build_geo_key(row):
        t = str(row["TYPE_2"]).strip()
        n = _re.sub(r"^(?:kabupaten|kota)\s+", "", str(row["NAME_2"])
        return norm(f"{t} {n}")

    gdf_tpt["join_key"] = gdf_tpt.apply(build_geo_key, axis=1)
```

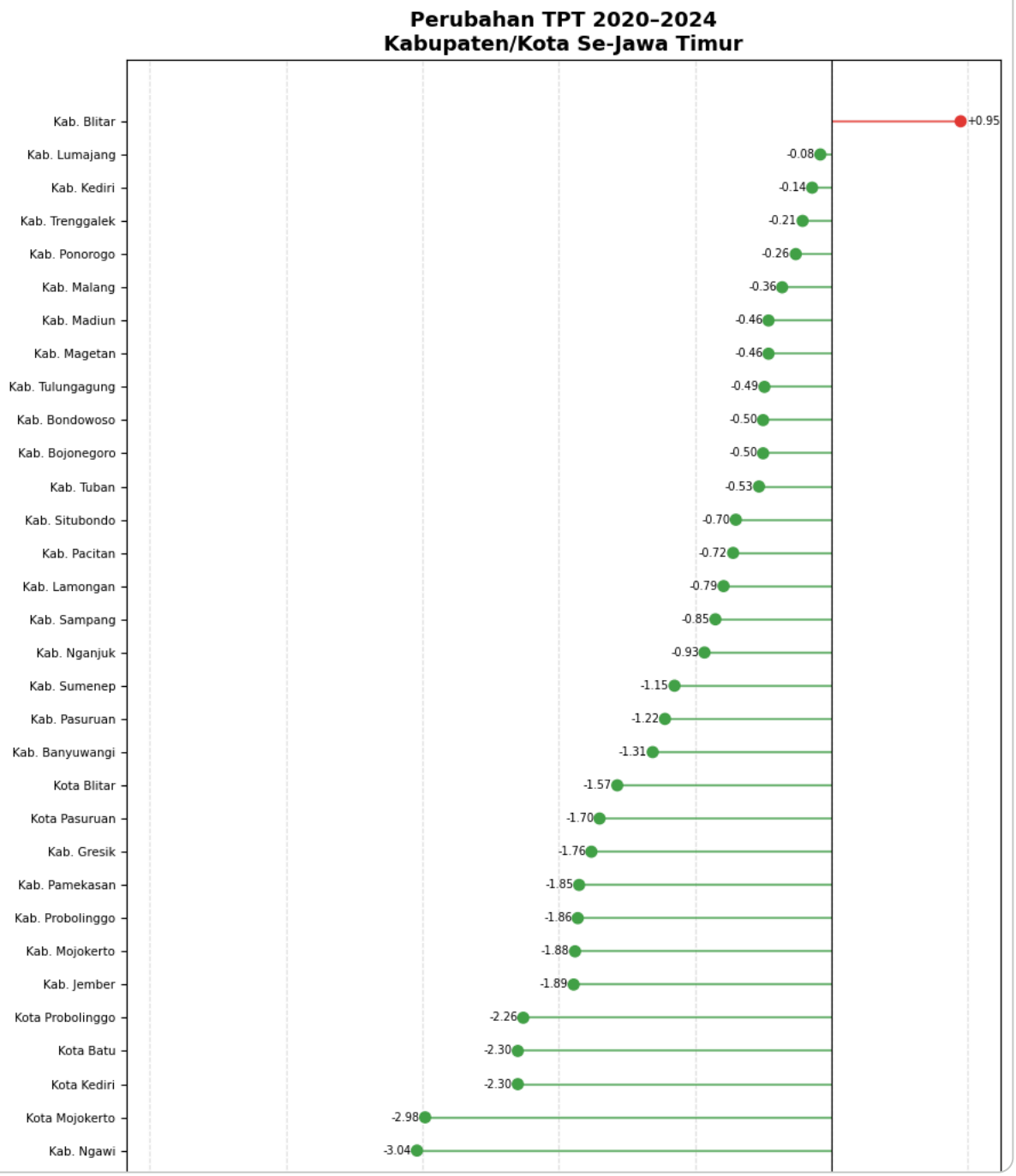
```
    df_tpt_choro = df_tpt[df_tpt["tahun"] == 2024].copy()
    df_tpt_choro["join_key"] = df_tpt_choro["region"].apply(norm)

    gdf_tpt_merged = gdf_tpt.merge(df_tpt_choro[["join_key", "TPT"]],
    matched = gdf_tpt_merged["TPT"].notna().sum()
    print(f"Match: {matched}/{len(gdf_tpt_merged)}")

    fig, ax = plt.subplots(1, 1, figsize=(14, 8))
    gdf_tpt_merged.plot(
        column="TPT",
        cmap="RdYlGn_r",
        linewidth=0.4,
        edgecolor="black",
        legend=True,
        legend_kwds={"label": "TPT (%)", "orientation": "vertical", "
        missing_kwds={"color": "lightgrey", "label": "Data NA"},
        ax=ax,
    )

    for _, row in gdf_tpt_merged.iterrows():
        if pd.notna(row.get("TPT")):
            cx, cy = row.geometry.centroid.x, row.geometry.centroid.y
            ax.text(cx, cy, f"{row['TPT']:.1f}", fontsize=5.5,
                    ha="center", va="center", color="black")

    ax.set_title("Peta Choropleth TPT 2024\nKabupaten/Kota Se-Jawa Ti
                 fontsize=14, fontweight="bold")
    ax.set_axis_off()
    plt.tight_layout()
    plt.savefig("tpt_choropleth_2024.png", dpi=150, bbox_inches="tigh
    plt.show()
    print("Saved: tpt_choropleth_2024.png")

except ImportError:
    print("geopandas tidak terpasang.")
```

```
Match: 38/38
```




```
# ── 6. Perubahan TPT 2020→2024 (Lollipop Chart) ──────────────────────
df_tpt_delta = df_tpt.pivot(index="region_short", columns="tahun", va
df_tpt_delta = df_tpt_delta[[2020, 2024]].dropna()
df_tpt_delta["delta"] = df_tpt_delta[2024] - df_tpt_delta[2020]
df_tpt_delta = df_tpt_delta.sort_values("delta", ascending=True)

# Negative = penurunan pengangguran (hijau/baik), positif = kenaikan
colors_tpt_delta = ["#43a047" if d < 0 else "#e53935" for d in df_tpt_

fig, ax = plt.subplots(figsize=(9, 13))
y_pos = range(len(df_tpt_delta))

ax.hlines(y_pos, 0, df_tpt_delta["delta"].values, colors=colors_tpt_d
ax.scatter(df_tpt_delta["delta"].values, y_pos, color=colors_tpt_delt
ax.axvline(0, color="black", linewidth=0.8)

ax.set_yticks(list(y_pos))
ax.set_yticklabels(df_tpt_delta.index, fontsize=7.5)
ax.set_xlabel("Perubahan TPT (poin persentase)", fontsize=11)
ax.set_title("Perubahan TPT 2020–2024\nKabupaten/Kota Se-Jawa Timur",
             fontsize=13, fontweight="bold")
ax.xaxis.set_minor_locator(mticker.AutoMinorLocator(5))
ax.grid(axis="x", which="major", linestyle="--", alpha=0.35)

for val, yp in zip(df_tpt_delta["delta"].values, y_pos):
    offset = 0.04 if val >= 0 else -0.04
    ha = "left" if val >= 0 else "right"
    ax.text(val + offset, yp, f"{val:+.2f}", va="center", ha=ha, font

legend_els = [
    Patch(facecolor="#43a047", label="Turun (Membaik)"),
    Patch(facecolor="#e53935", label="Naik (Memburuk)"),
]
ax.legend(handles=legend_els, fontsize=9, loc="lower right")
plt.tight_layout()
```

```
plt.savefig("tpt_delta_2020_2024.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: tpt_delta_2020_2024.png")
```



Visualisasi Pendapatan Asli Daerah (PAD) Jawa Timur 2020–2024

```
import re
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import matplotlib.ticker as mticker
from matplotlib.patches import Patch

warnings.filterwarnings("ignore")

# ── 1. Load PAD panel ──────────────────────────────────────────────
PAD_ROOT = Path(".").resolve() / "pad"

# Target rows (normalised lowercase match)
TARGET_ROWS = {
    "pad":                    "PAD_total",
    "pajak daerah":           "Pajak_Daerah",
    "retribusi daerah":       "Retribusi_Daerah",
    "hasil pengelolaan kekayaan daerah yang dipisahkan": "Kekayaan_Di
    "lain-lain pad yang sah": "LainLain_PAD",
}

def folder_to_region(folder_name: str) -> str:
    if folder_name.startswith("kab_"):
        base = folder_name[4:].replace("_", " ").title()
        return "Kabupaten " + base
    if folder_name.startswith("kota_"):
        base = folder_name[5:].replace("_", " ").title()
        return "Kota " + base
    return folder_name.replace("_", " ").title()

def clean_num(v):
    if pd.isna(v): return np.nan
    s = str(v).strip().replace(" ", "").replace(",", ".")
    try: return float(s)
    except: return np.nan

def load_pad(pad_root: Path) -> pd.DataFrame:
    records = []
    for folder in sorted(pad_root.iterdir()):
        if not folder.is_dir():
            continue
        region = folder_to_region(folder.name)
        for yr in range(2020, 2025):
            # Filenames: try folder_name first, then bare base name
            candidates = list(folder.glob(f"*{yr}.xlsx"))
            if not candidates:
                continue
            path = candidates[0]
            try:
                df = pd.read_excel(path, header=None)
            except Exception:
                continue

            row_data = {"region": region, "tahun": yr}
            for _, row in df.iterrows():
                akun = str(row.iloc[0]).strip().lower() if pd.notna(r
                for key, col_name in TARGET_ROWS.items():
                    if akun == key:
                        row_data[col_name + "_anggaran"]  = clean_num
```

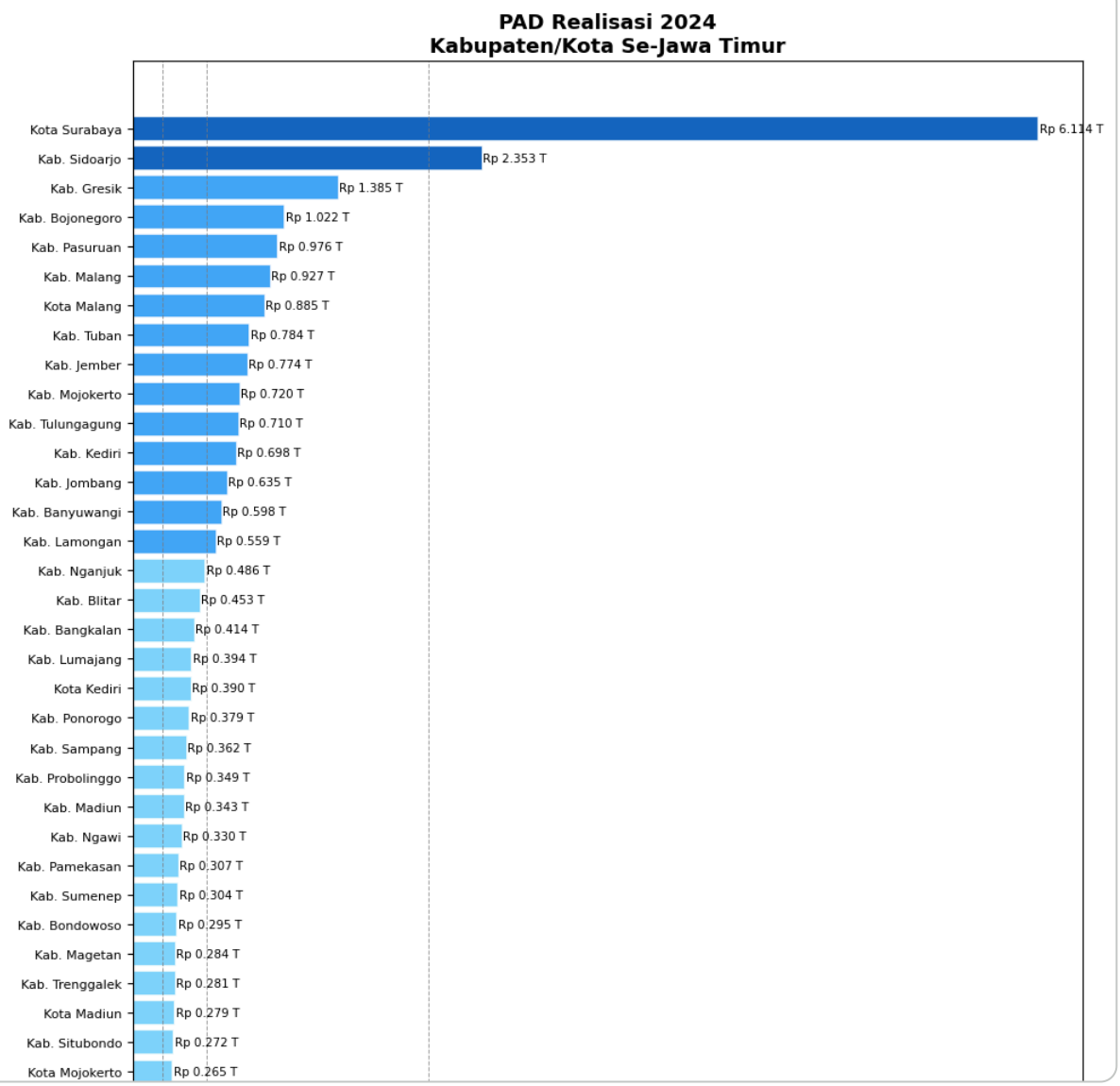
```
                        row_data[col_name + "_realisasi"] = clean_num
            records.append(row_data)

    df_pad = pd.DataFrame(records)
    df_pad["region_short"] = (
        df_pad["region"]
        .str.replace(r"^Kabupaten\s+", "Kab. ", regex=True)
        .str.replace(r"^Kota\s+",      "Kota ", regex=True)
    )
    return df_pad

df_pad = load_pad(PAD_ROOT)

# Convert to triliun for readability
TRILUN = 1e12
for col in df_pad.columns:
    if col not in ("region", "tahun", "region_short"):
        df_pad[col] = df_pad[col] / TRILUN

print(f"Wilayah: {df_pad['region'].nunique()}, Tahun: {sorted(df_pad[
print(f"Null PAD_total_realisasi: {df_pad['PAD_total_realisasi'].isna
display(df_pad[["region_short","tahun","PAD_total_realisasi","Pajak_D
                "Retribusi_Daerah_realisasi","Kekayaan_Dipisahkan_rea
```

```
Wilayah: 38, Tahun: [np.int64(2020), np.int64(2021), np.int64(2022), n
Null PAD_total_realisasi: 0
```

	region_short	tahun	PAD_total_realisasi	Pajak_Daerah_realisasi	Re
0	Kab. Bangkalan	2020	0.233178	0.046624	
1	Kab. Bangkalan	2021	0.271657	0.048377	
2	Kab. Bangkalan	2022	0.281897	0.056255	
3	Kab. Bangkalan	2023	0.353692	0.060387	
4	Kab. Bangkalan	2024	0.414116	0.071755	
5	Kab. Banyuwangi	2020	0.482740	0.191631	
6	Kab. Banyuwangi	2021	0.520022	0.206146	
7	Kab. Banyuwangi	2022	0.516757	0.231459	
8	Kab. Banyuwangi	2023	0.507674	0.256037	
9	Kab. Banyuwangi	2024	0.597544	0.298505	

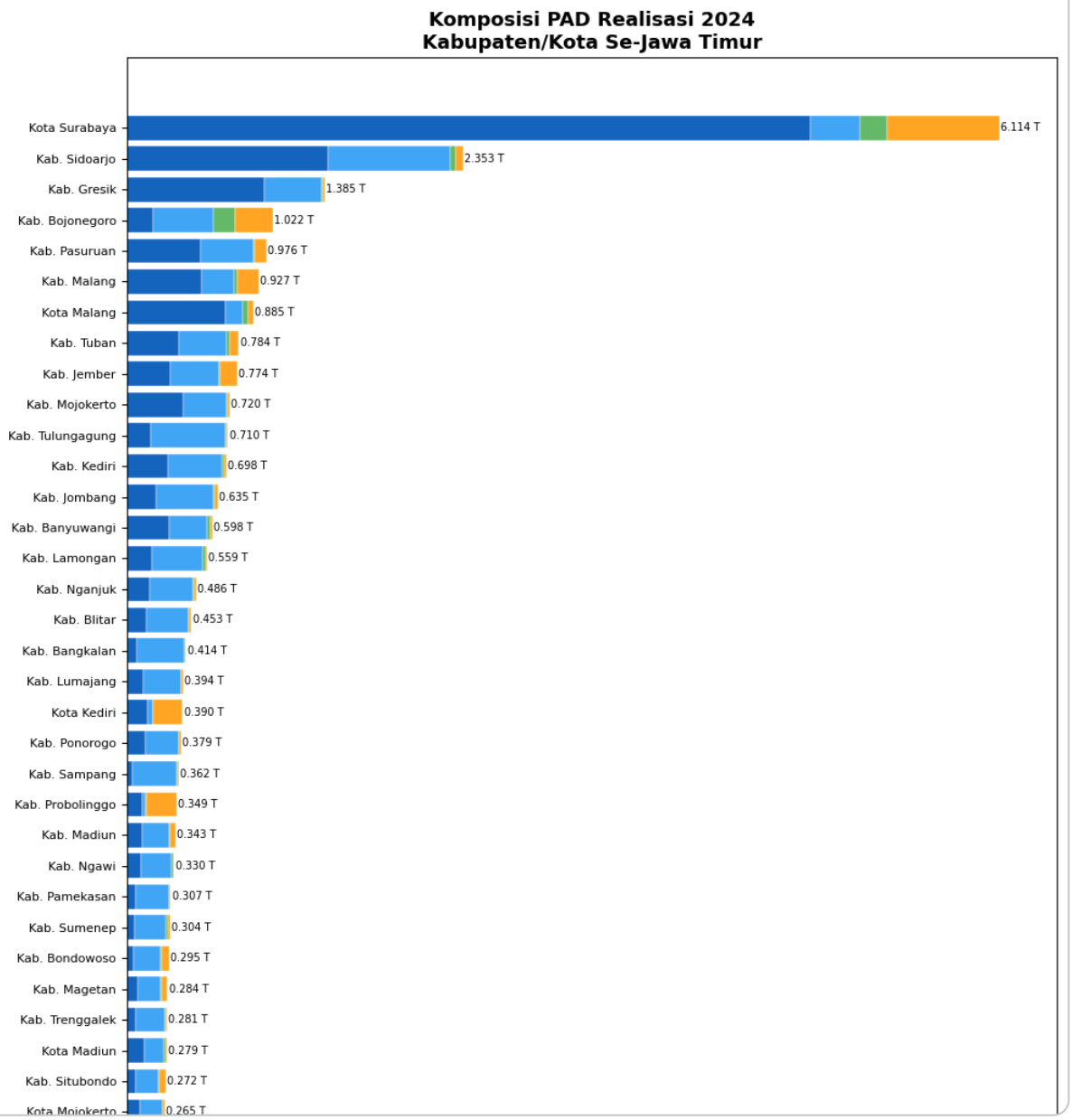
```
# ── 2. Bar Chart: PAD Realisasi 2024 (sorted) ──────────────────────
df_pad_2024 = df_pad[df_pad["tahun"] == 2024].sort_values("PAD_total_
```

```
def pad_color(val):  # val in triliun
    if val >= 2:    return "#1565c0"
    if val >= 0.5:  return "#42a5f5"
    if val >= 0.2:  return "#81d4fa"
    return "#b3e5fc"

colors_pad = [pad_color(v) for v in df_pad_2024["PAD_total_realisasi"

fig, ax = plt.subplots(figsize=(10, 12))
bars = ax.barh(df_pad_2024["region_short"], df_pad_2024["PAD_total_rea
               color=colors_pad, edgecolor="white", linewidth=0.4)

for bar, val in zip(bars, df_pad_2024["PAD_total_realisasi"]):
    ax.text(bar.get_width() + 0.01, bar.get_y() + bar.get_height() / 2
            f"Rp {val:.3f} T", va="center", ha="left", fontsize=7.5)

for ref, lbl in [(0.2, "0,2 T"), (0.5, "0,5 T"), (2, "2 T")]:
    ax.axvline(ref, color="grey", linewidth=0.7, linestyle="--", alpha
    ax.text(ref + 0.01, 0, lbl, fontsize=7, color="grey", va="bottom"

ax.set_xlabel("PAD Realisasi (Triliun Rp)", fontsize=11)
ax.set_xlim(0, df_pad_2024["PAD_total_realisasi"].max() + 0.3)
ax.set_title("PAD Realisasi 2024\nKabupaten/Kota Se-Jawa Timur",
             fontsize=13, fontweight="bold")
ax.tick_params(axis="y", labelsize=8)

legend_els = [
    Patch(facecolor="#1565c0", label="≥ 2 T"),
    Patch(facecolor="#42a5f5", label="0,5–2 T"),
    Patch(facecolor="#81d4fa", label="0,2–0,5 T"),
    Patch(facecolor="#b3e5fc", label="< 0,2 T"),
]
ax.legend(handles=legend_els, fontsize=8, loc="lower right")
plt.tight_layout()
plt.savefig("pad_bar_2024.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: pad_bar_2024.png")
```




```
# ── 3. Stacked Bar: Komposisi PAD 2024 (top 20 + lainnya) ──────────
comp_cols = ["Pajak_Daerah_realisasi", "Retribusi_Daerah_realisasi",
             "Kekayaan_Dipisahkan_realisasi", "LainLain_PAD_realisasi"
comp_labels = ["Pajak Daerah", "Retribusi Daerah", "Kekayaan Dipisahka
comp_colors = ["#1565c0", "#42a5f5", "#66bb6a", "#ffa726"]

df_comp = df_pad[df_pad["tahun"] == 2024].copy()
df_comp = df_comp.sort_values("PAD_total_realisasi", ascending=True)

fig, ax = plt.subplots(figsize=(10, 13))
left = np.zeros(len(df_comp))

for col, lbl, clr in zip(comp_cols, comp_labels, comp_colors):
    vals = df_comp[col].fillna(0).values
    ax.barh(df_comp["region_short"], vals, left=left, color=clr,
            label=lbl, edgecolor="white", linewidth=0.3)
    left += vals

# Total label
for i, (total, region) in enumerate(zip(df_comp["PAD_total_realisasi"
    ax.text(total + 0.01, i, f"{total:.3f} T", va="center", ha="left"

ax.set_xlabel("PAD Realisasi (Triliun Rp)", fontsize=11)
```

```
ax.set_title("Komposisi PAD Realisasi 2024\nKabupaten/Kota Se-Jawa Ti
             fontsize=13, fontweight="bold")
ax.tick_params(axis="y", labelsize=8)
ax.legend(fontsize=8, loc="lower right")
ax.set_xlim(0, df_comp["PAD_total_realisasi"].max() + 0.4)
plt.tight_layout()
plt.savefig("pad_stacked_2024.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: pad_stacked_2024.png")
```




```
# ── 4. Line Chart: Tren PAD Realisasi 2020–2024 ──────────────────
pivot_pad = df_pad.pivot(index="region_short", columns="tahun", value
pivot_pad = pivot_pad.sort_values(2024, ascending=False)
years_pad = list(range(2020, 2025))

fig, ax = plt.subplots(figsize=(13, 8))
cmap_pad = plt.cm.get_cmap("tab20", len(pivot_pad))

for i, (region, row) in enumerate(pivot_pad.iterrows()):
    vals = row[years_pad].values
```